day03_Python基础_基础语法

今日内容:

- 1- Python基本概念 【了解】
- 2- Python的安装操作: 【参考课件完成】
  - Python的解析器安装
  - Python的开发工具安装
- 3- Python的基础语法: 【实操 -- 记忆】
  - 3.1 注释
  - 3.2 变量
  - 3.3 输入和输出
  - 3.4 数据类型转换

1.【了解】Python概述与安装

1.1 为什么要学习Python

① 技术趋势

Python自带明星属性，热度稳居编程语言界前三



https://www.tiobe.com/tiobe-index/

https://pypl.github.io/PYPL.html

② 简单易学

开发代码少，精确表达需求逻辑；**33个关键字，7种基本数据类型**；语法规则简单，接近自然语言。



③ 应用广泛

Python语言涉及IT行业70%以上的技术领域



1.2 Python语言的诞生

1989年，为了打发圣诞节假期，龟叔(吉多·范·罗苏姆)开始写Python语言的编译器；
1991年，第一个Python编译器诞生
Python这个名字，来自龟叔所挚爱的电视剧Monty Python's Flying Circus (蒙蒂·蟒蛇的飞行马戏团)



1.3 python语言的优缺点

优点

简单：Python是一种代表简单主义思想的语言。阅读一个良好的Python程序就感觉像是在读英语一样，Python的这种代码本质是它最大的优点之一。它使你能够专注于解决问题而不是去搞明白语言本身。

易学：就如同你即将看到的一样，Python极其容易上手。前面已经提到了，Python有极其简单的语法。

免费、开源：Python开源的。简单地说，你可以自由地阅读它的源代码、对它做改动、这是为什么Python如此优秀的原因之一，它是由一群希望看到一个更加优秀的Python的人创造并经常改进着的。

可移植性：由于它的开源本质，Python已经被移植在许多平台上（经过改动使它能够工作在不同平台上）。如果你小心地避免使用依赖于系统的特性，那么你的所有Python程序无需修改就可以在下述任何平台上面运行。

丰富的库：Python标准库确实很庞大。它可以帮助你处理各种工作，包括正则表达式、文档生成、单元测试、线程、数据库、网页浏览器、CGI、FTP、电子邮件、XML、XML-RPC、HTML、WAV文件、密码系统、GUI（图形用户界面）、Tk和其他与系统有关的操作。记住，只要安装了Python，所有这些功能都是可用的。这被称作Python的“功能齐全”理念。

缺点

Python语言非常完善，没有明显的短板和缺点，唯一的缺点就是执行效率慢，这个是解释型语言所通有的，同时这个缺点也将被计算机越来越强大的性能所弥补。

1.4 Python解析器的作用

demo.py

```
print('Hello World')
```

由于Python属于高级语言，其并不能直接在计算机中运行，因为缺少Python语言的运行环境：Python解析器



Python解析器的作用：就是把Python代码转换为计算机底层可以识别的机器语言，如0101...

1.5 Python解析器的种类

① CPython，C语言开发的解释器[官方]，应用广泛的解释器。

② IPython，基于CPython的一种交互式解释器。

③ 其他解释器

PyPy，基于Python语言开发的解释器。

JPython，运行在Java平台的解释器，直接把Python代码编译成Java字节码执行。

IronPython，运行在微软.Net平台上的Python解释器，可直接把Python代码编译成.Net的字节码。

1.6 Python解释器版本

在生产环境中，我们一般不会选择最新版本的Python，因为可能会存在未知Bug，所以一般强烈建议大家在选择软件版本时,选择市场常用的软件版本

Python3.8及以上

... Python3.8、Python3.9、Python3.10、**Python3.11**、Python3.12、Python3.13 ...

原生版本下载地址：https://www.python.org/downloads

1.7 Anaconda安装Python解析器

1.7.1 简介

Anaconda 是一个开源的 **Python/R 数据科学平台**，它极大地简化了包管理和环境管理。你可以把它理解为一个“超级工具箱”，里面不仅包含了Python本身，还预装了超过1500个用于数据科学（如数据分析、机器学习、可视化）的流行库和工具。

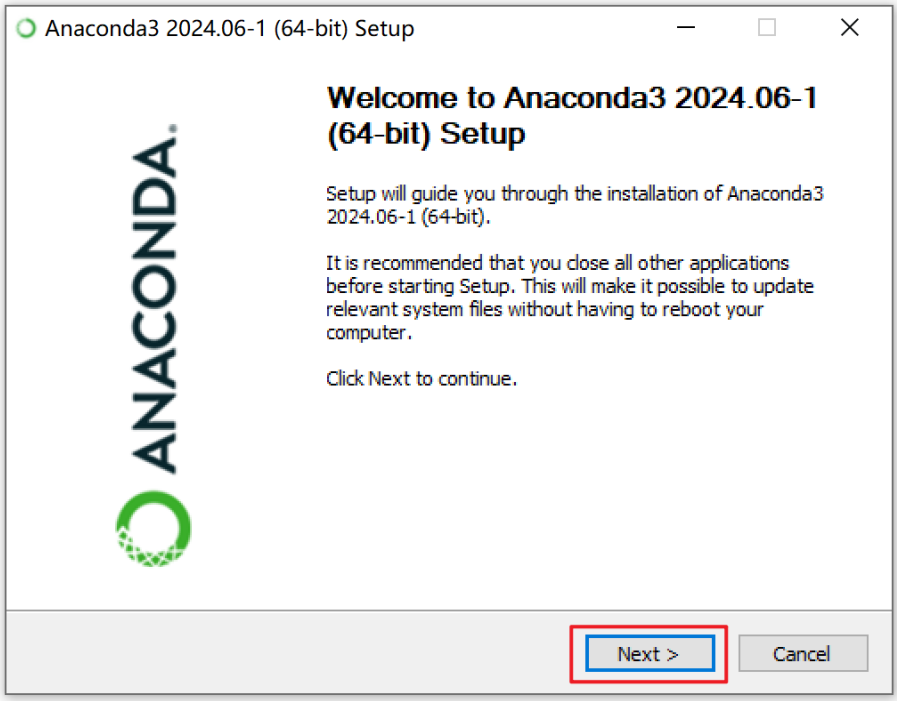
此外，大家要清楚Anaconda, Miniconda 和 Conda 的关系与区别

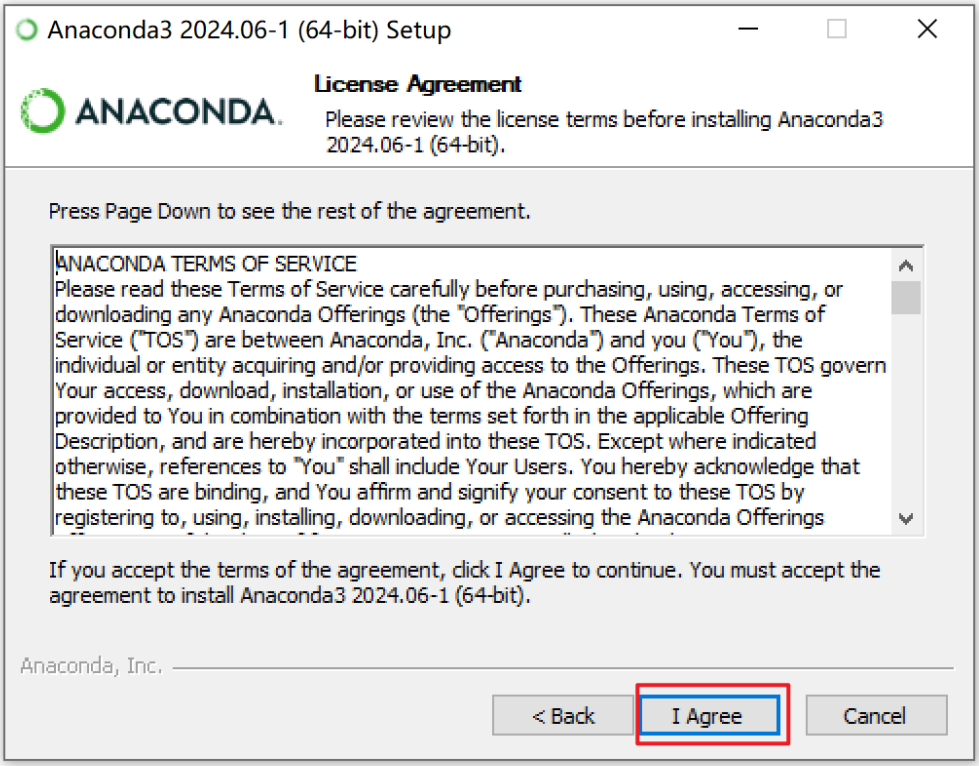
- **Conda：** 是底层的**包管理和环境管理工具本身**。它是一个独立的命令行工具。
- **Anaconda：** 是一个“全家桶”发行版。它**包含了Conda、Python、1500+个数据科学库以及Navigator等图形化工具**。体积较大（约500MB-1GB）。
- **Miniconda：** 是一个“迷你版”的Anaconda。它**只包含Conda、Python和少量核心依赖**。体积非常小。

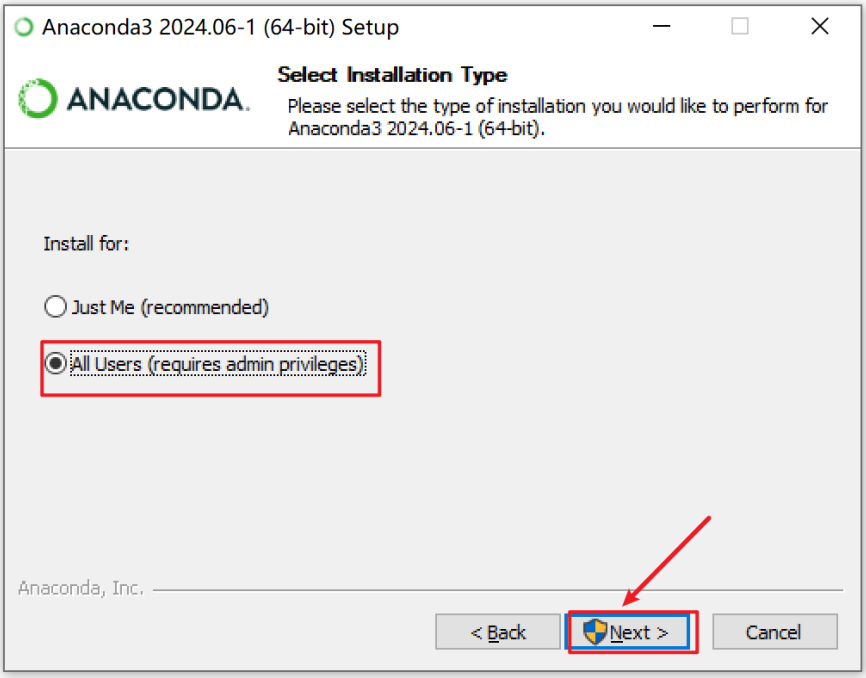
如何选择?

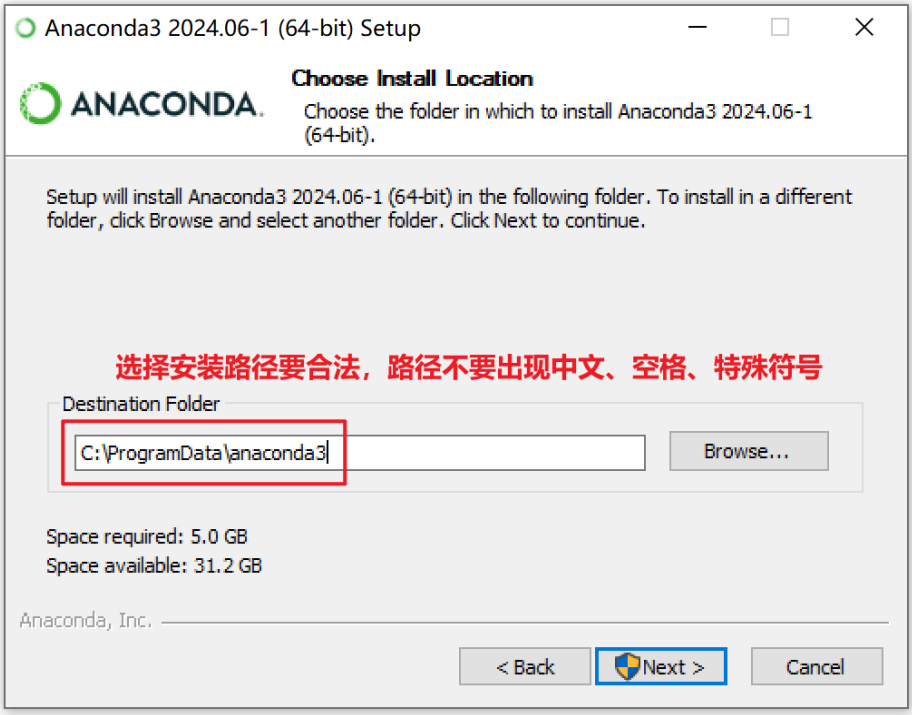
- **初学者/希望快速上手的用户：** 推荐直接安装 **Anaconda**。省时省力，无需担心初始配置。
- **高级用户/磁盘空间有限/追求定制化：** 推荐安装 **Miniconda**。你可以从一个干净的基础开始，只安装你需要的包，从而保持环境的精简。

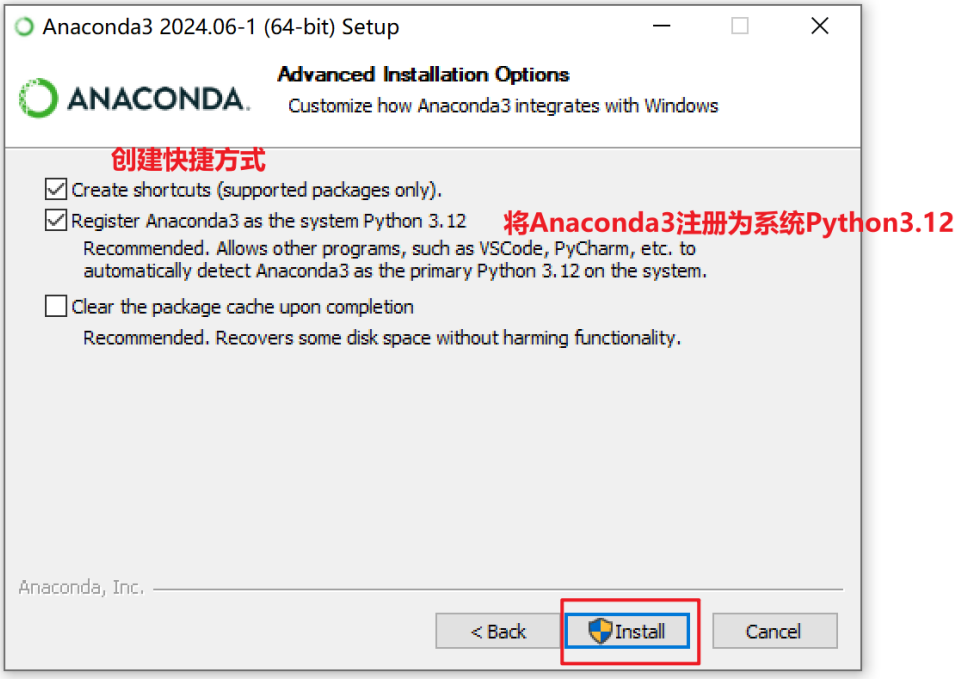
1.7.2 安装

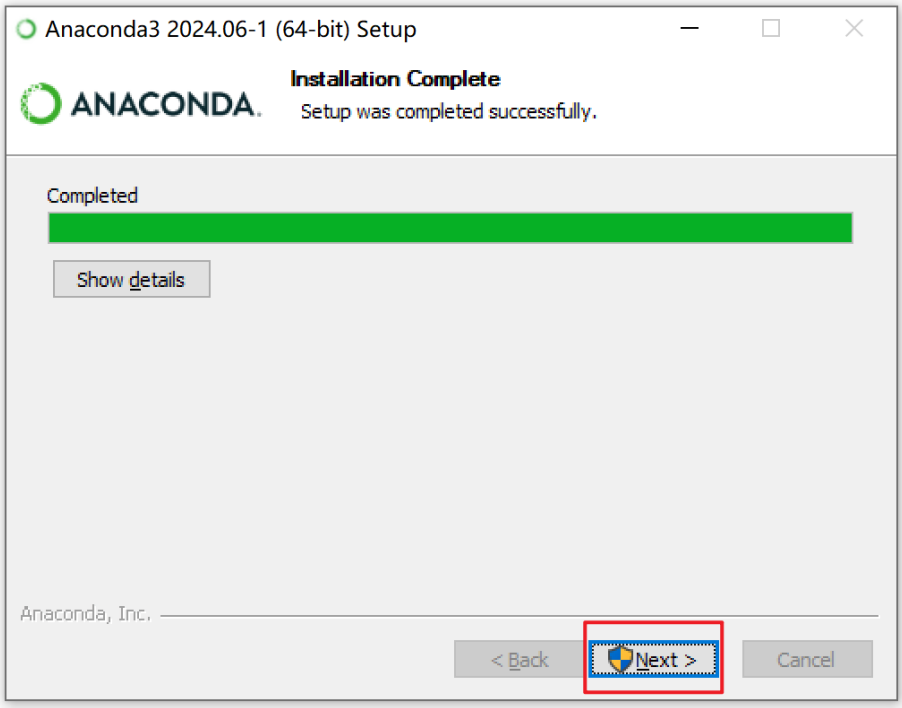


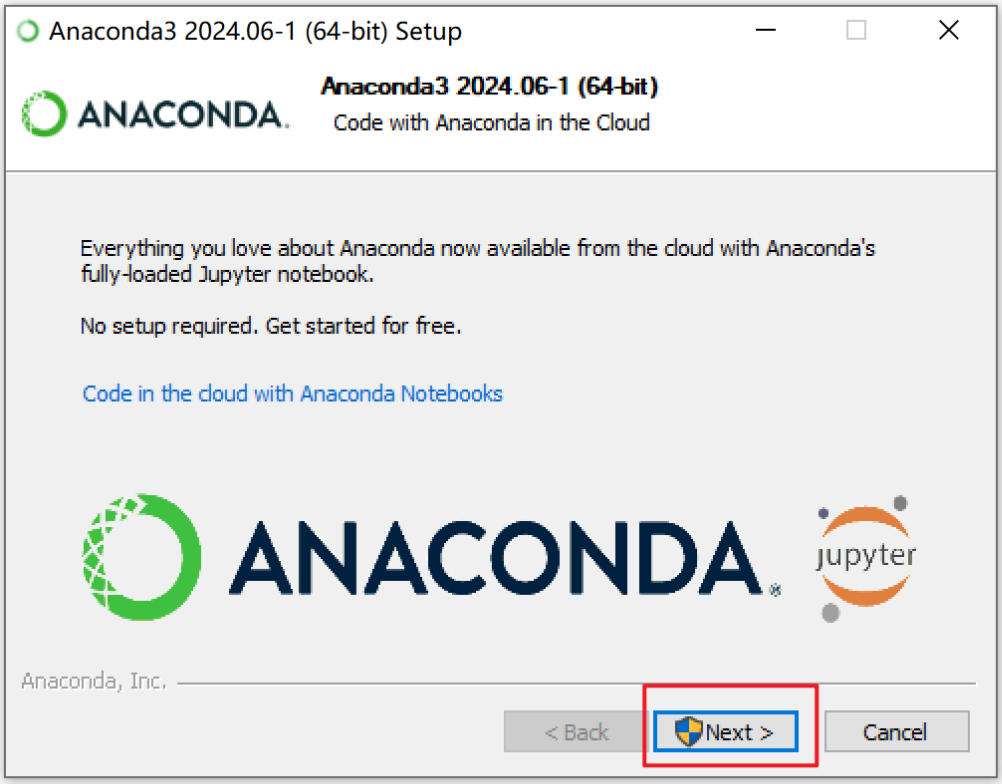

Anaconda3 2024.06-1 (64-bit) Setup
ANACONDA.
License Agreement
Please review the license terms before installing Anaconda3 2024.06-1 (64-bit).
Press Page Down to see the rest of the agreement.
ANACONDA TERMS OF SERVICE
Please read these Terms of Service carefully before purchasing, using, accessing, or downloading any Anaconda Offerings (the "Offerings"). These Anaconda Terms of Service ("TOS") are between Anaconda, Inc. ("Anaconda") and you ("You"), the individual or entity acquiring and/or providing access to the Offerings. These TOS govern Your access, download, installation, or use of the Anaconda Offerings, which are provided to You in combination with the terms set forth in the applicable Offering Description, and are hereby incorporated into these TOS. Except where indicated otherwise, references to "You" shall include Your Users. You hereby acknowledge that these TOS are binding, and You affirm and signify your consent to these TOS by registering to, using, installing, downloading, or accessing the Anaconda Offerings
If you accept the terms of the agreement, click I Agree to continue. You must accept the agreement to install Anaconda3 2024.06-1 (64-bit).
Anaconda, Inc.
< Back
I Agree
Cancel

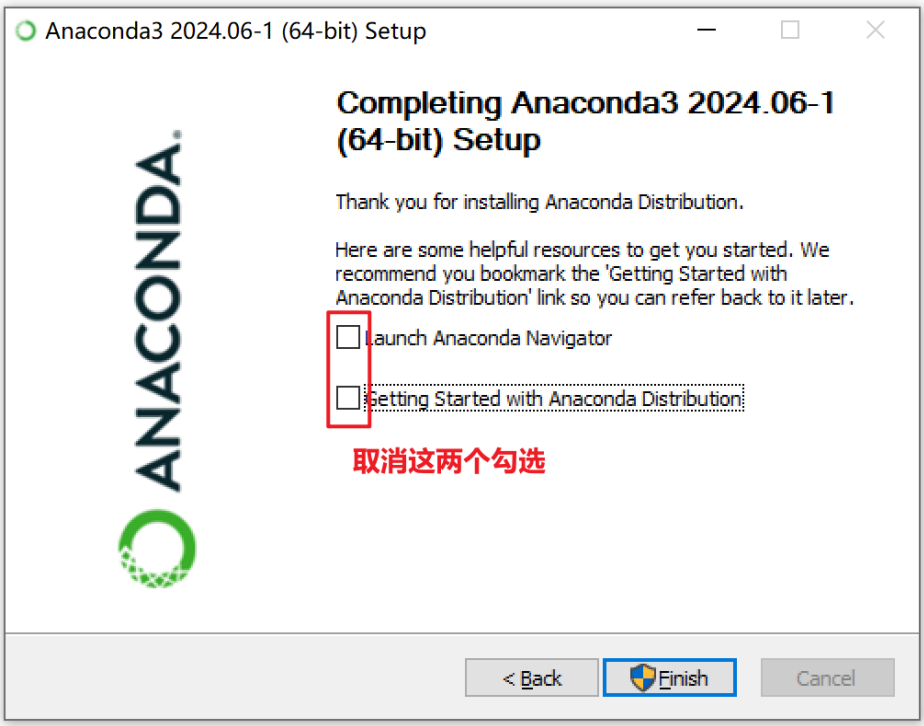

Anaconda3 2024.06-1 (64-bit) Setup
ANACONDA.
Select Installation Type
Please select the type of installation you would like to perform for Anaconda3 2024.06-1 (64-bit).
Install for:
Just Me (recommended)
All Users (requires admin privileges)
Anaconda, Inc.
< Back
Next >
Cancel


Anaconda3 2024.06-1 (64-bit) Setup
ANACONDA.
Choose Install Location
Choose the folder in which to install Anaconda3 2024.06-1 (64-bit).
Setup will install Anaconda3 2024.06-1 (64-bit) in the following folder. To install in a different folder, click Browse and select another folder. Click Next to continue.
选择安装路径要合法，路径不要出现中文、空格、特殊符号
Destination Folder
C:\ProgramData\anaconda3
Browse...
Space required: 5.0 GB
Space available: 31.2 GB
Anaconda, Inc.
< Back
Next >
Cancel


Anaconda3 2024.06-1 (64-bit) Setup
ANACONDA.
Advanced Installation Options
Customize how Anaconda3 integrates with Windows
创建快捷方式
Create shortcuts (supported packages only).
Register Anaconda3 as the system Python 3.12
将Anaconda3注册为系统Python3.12
Recommended. Allows other programs, such as VSCode, PyCharm, etc. to automatically detect Anaconda3 as the primary Python 3.12 on the system.
Clear the package cache upon completion
Recommended. Recovers some disk space without harming functionality.
Anaconda, Inc.
< Back
Install
Cancel

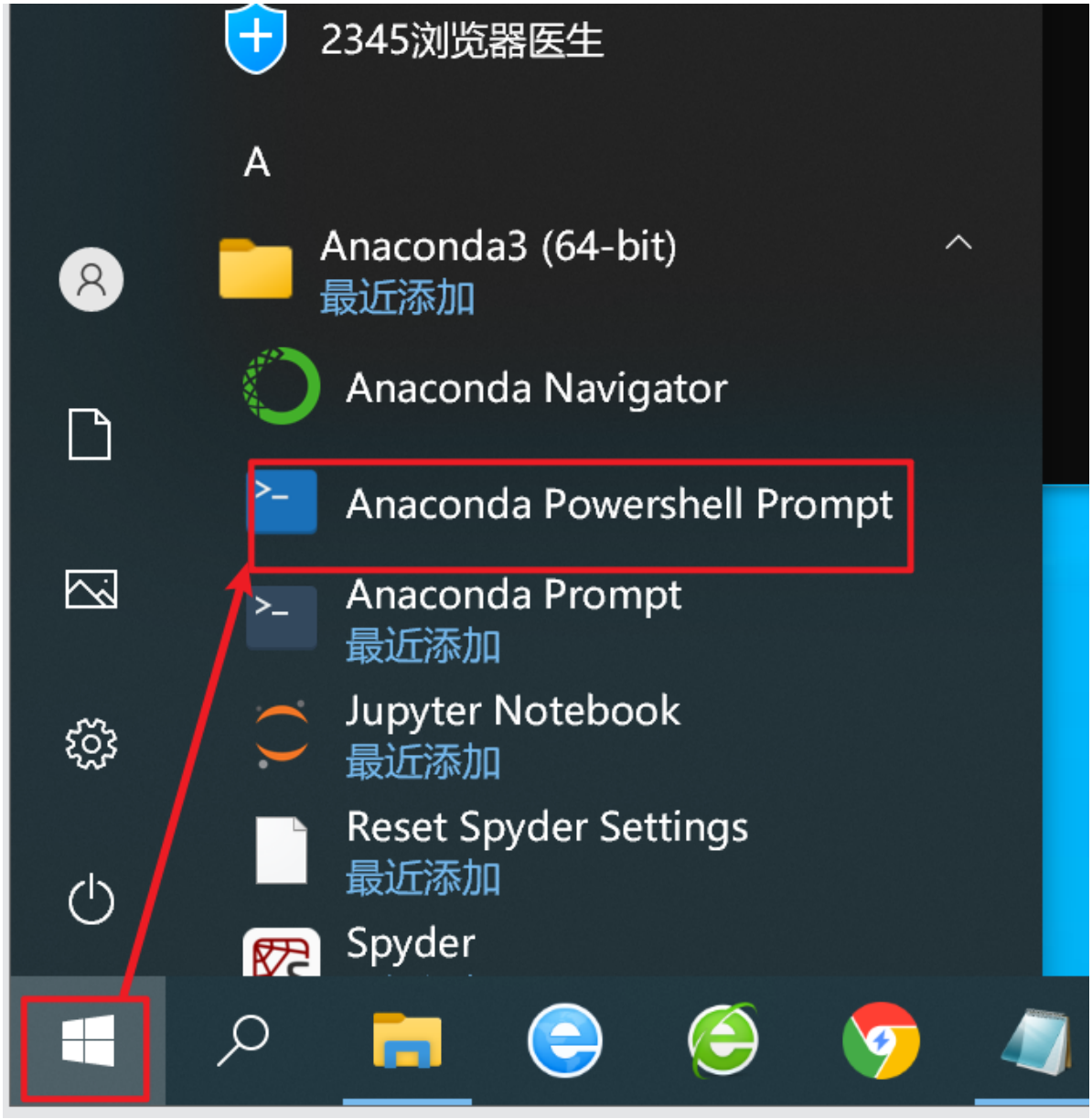

Anaconda3 2024.06-1 (64-bit) Setup
ANACONDA.
Installation Complete
Setup was completed successfully.
Completed
Show details
Anaconda, Inc.
< Back
Next >
Cancel


Anaconda3 2024.06-1 (64-bit) Setup
ANACONDA.
Anaconda3 2024.06-1 (64-bit)
Code with Anaconda in the Cloud
Everything you love about Anaconda now available from the cloud with Anaconda's fully-loaded Jupyter notebook.
No setup required. Get started for free.
Code in the cloud with Anaconda Notebooks
ANACONDA.
jupyter
Anaconda, Inc.
< Back
Next >
Cancel



1.7.3 测试

- 打开开始菜单点击Anaconda Powershell Prompt, 分别输入 **conda -V** 和 **python -V**, 能出现版本号, 即为安装成功.



管理员: Anaconda Powershell Prompt

```
(base) PS C:\Users\Administrator> conda -V
conda 24.5.0
(base) PS C:\Users\Administrator> python -V
Python 3.12.4
(base) PS C:\Users\Administrator>
```

2. 【实操】Python开发利器之PyCharm

2.1 为什么要安装PyCharm

工欲善其事必先利其器

在Python的开发领域，其开发工具非常非常多，EditPlus、Notepad++、Sublime Text3、Visual Studio Code、PyCharm（目前功能最强大的IDE）



2.2 PyCharm的主要作用

PyCharm是一种Python IDE （集成开发环境），带有一整套可以帮助用户在使用Python语言开发时提高其效率的工具，内部集成的功能如下：

Project管理

智能提示

语法高亮

代码跳转

调试代码

解释代码(解释器)

框架和库

……

2.3 PyCharm的分类

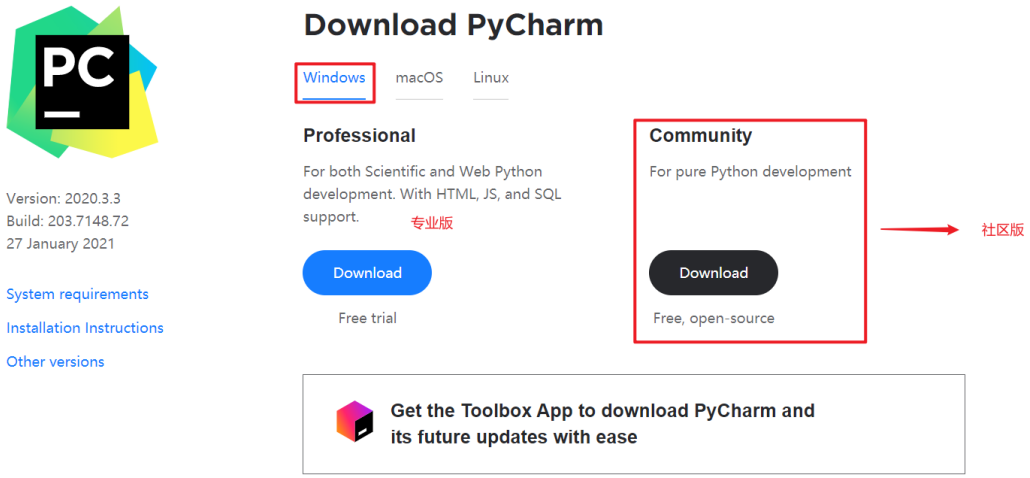
PyCharm一共有两个版本：专业版（收费） 与 社区版（免费、开源）

	Professional Edition	Community Edition
Intelligent Python editor	✓	✓
Graphical debugger and test runner	✓	✓
Navigation and Refactorings	✓	✓
Code inspections	✓	✓
VCS support	✓	✓
Scientific tools	✓	
Web development	✓	
Python web frameworks	✓	
Python Profiler	✓	
Remote development capabilities	✓	
Database & SQL support	✓	

在基础班，PyCharm社区版足够我们使用，绰绰有余。

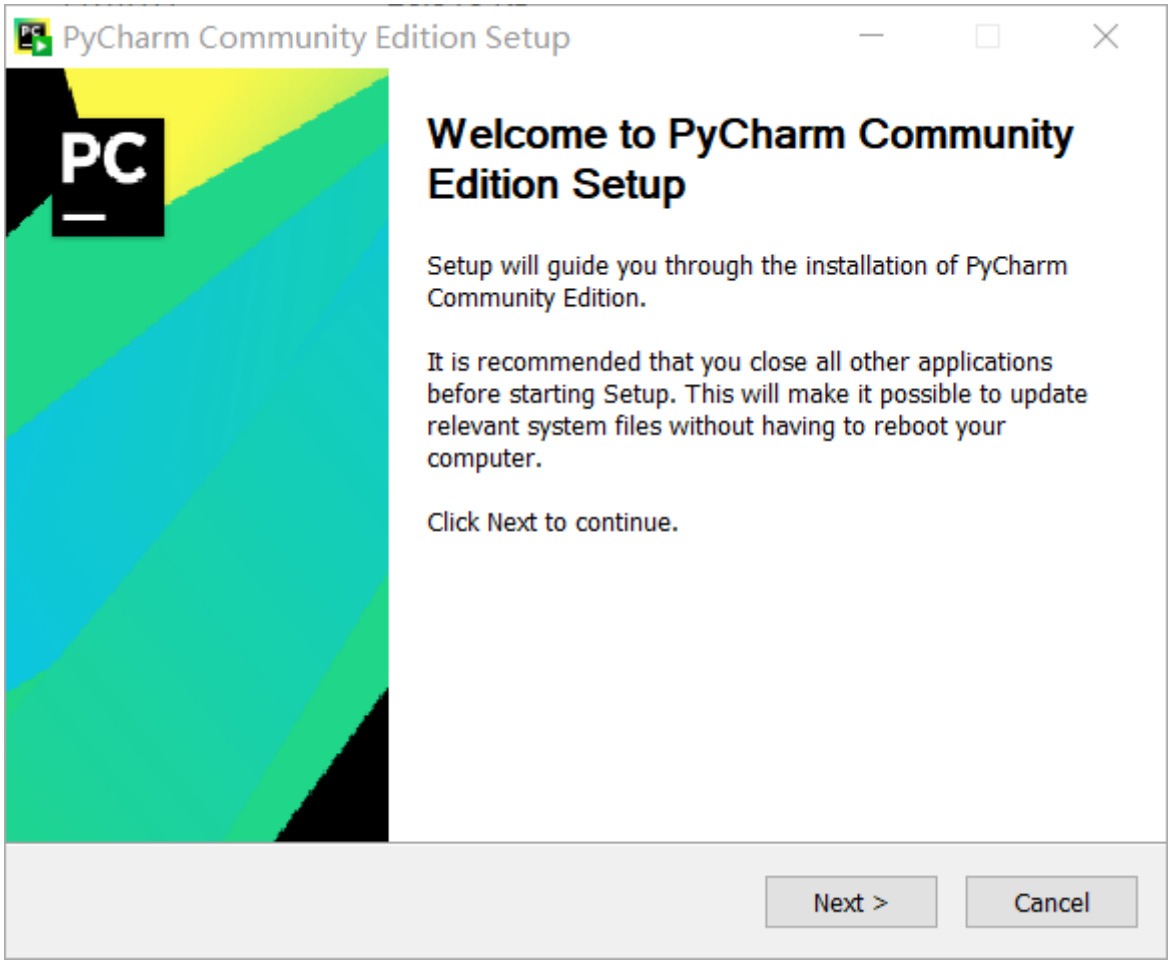
2.4 下载PyCharm

下载地址：https://www.jetbrains.com/pycharm/download/#section=windows

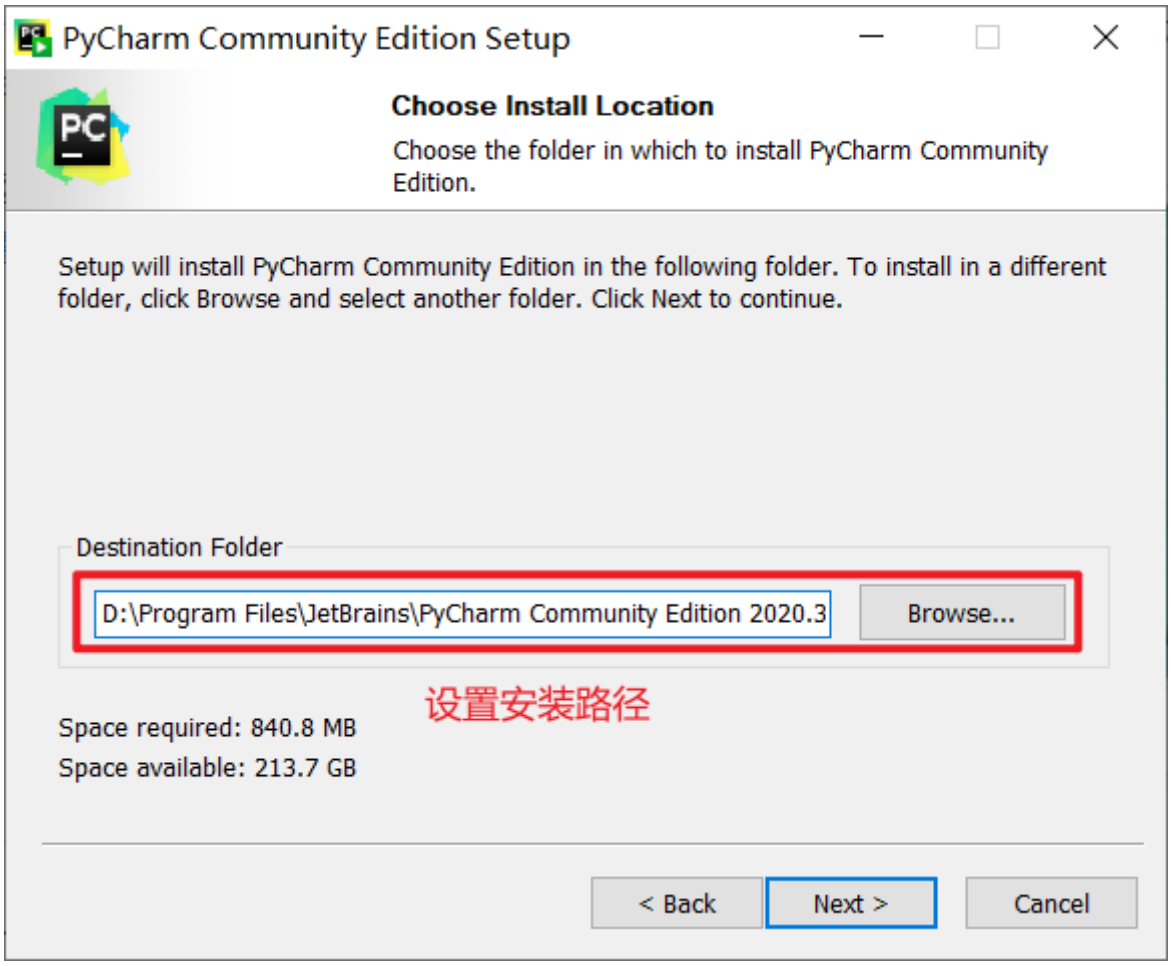


2.5 PyCharm安装

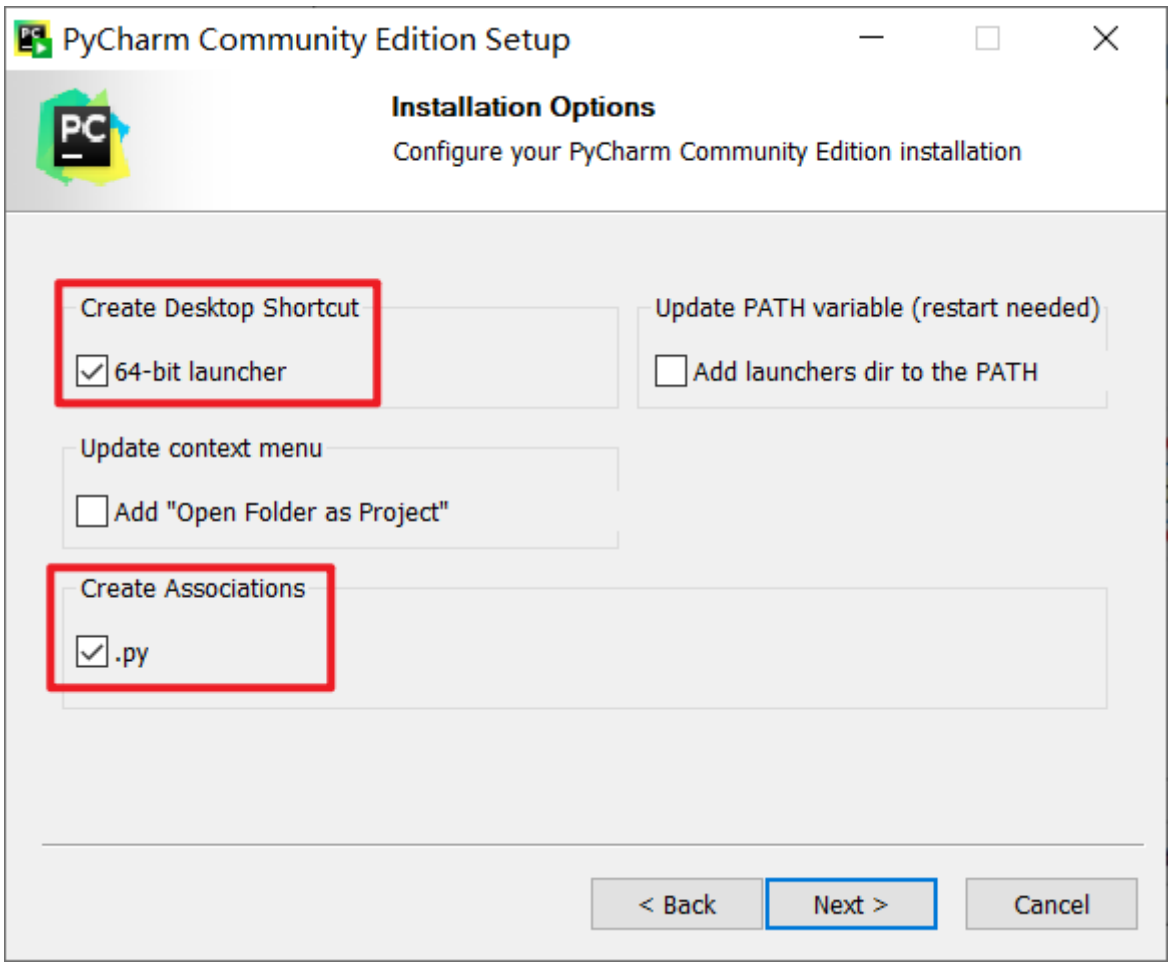
第一步：双击PyCharm软件安装包，进行软件安装



第二步：设置软件的安装路径，理论上没有任何要求，但是建议放在除C盘以外的盘符



第三步：PyCharm基本设置，创建桌面图标与.py文件关联

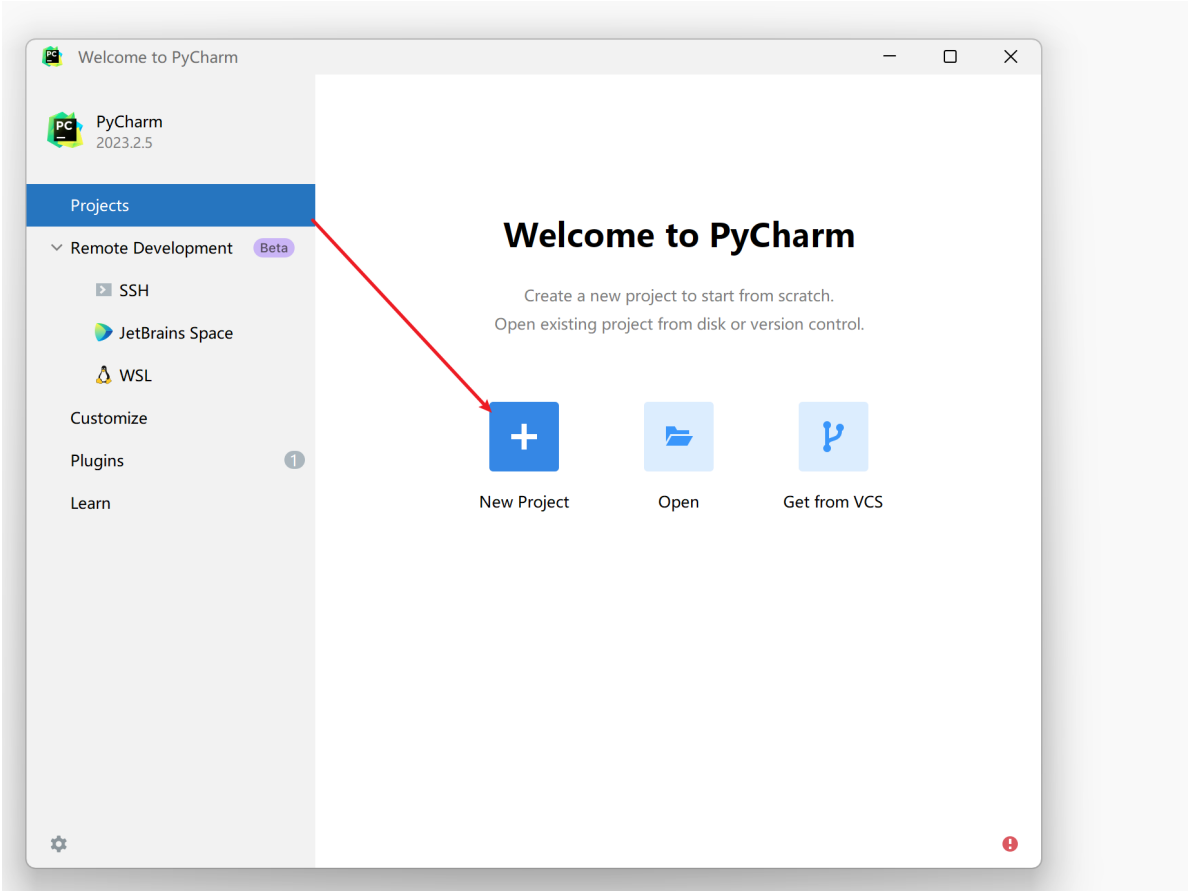


2.6 PyCharm软件的使用（更改）

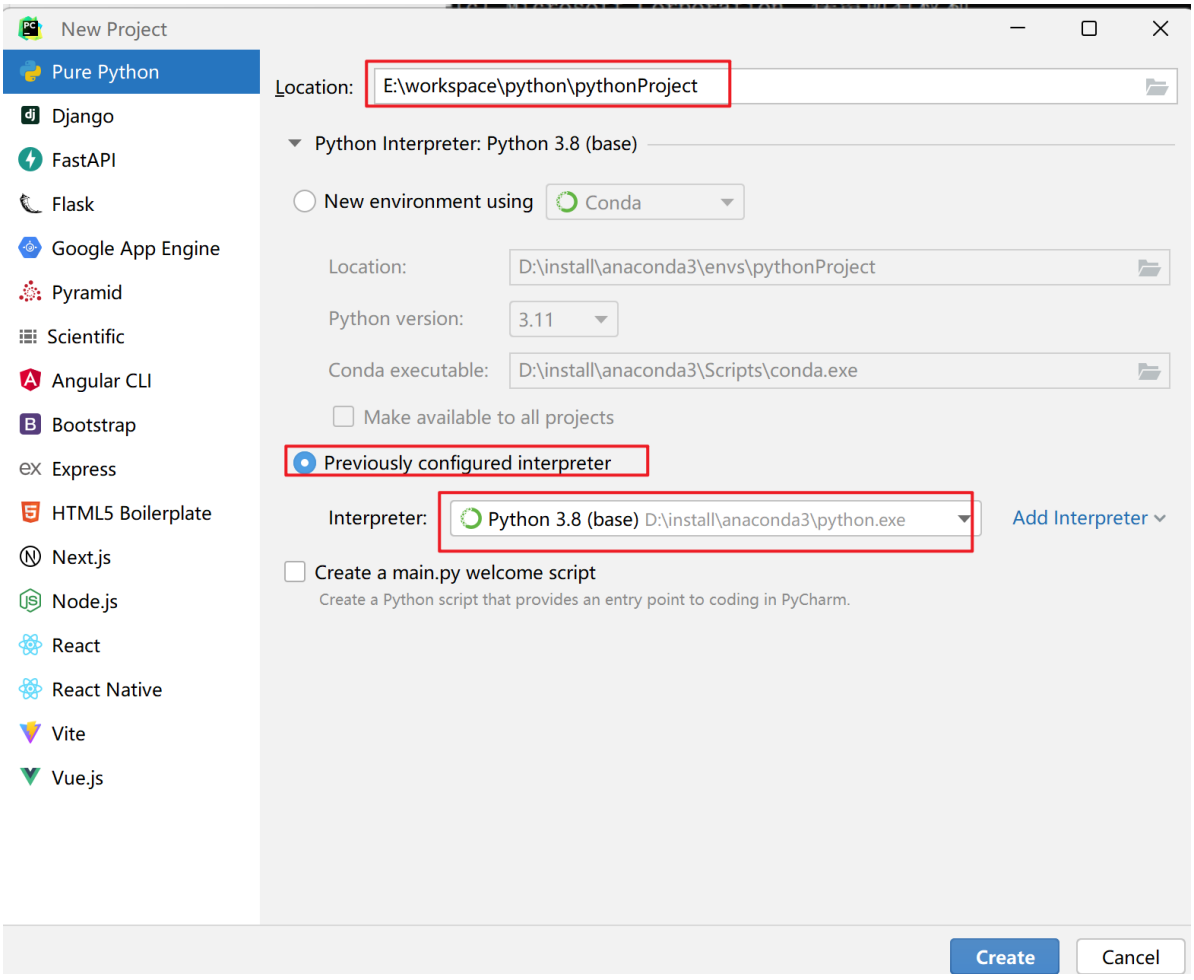
☆ 创建Python项目（重新截图2023.2.5 白色风格）

什么是项目？其实我们在实际开发中，每次参与一个工作的开发都是一个项目的开发过程。所以使用PyCharm的第一件事就是学习Python项目的创建过程。

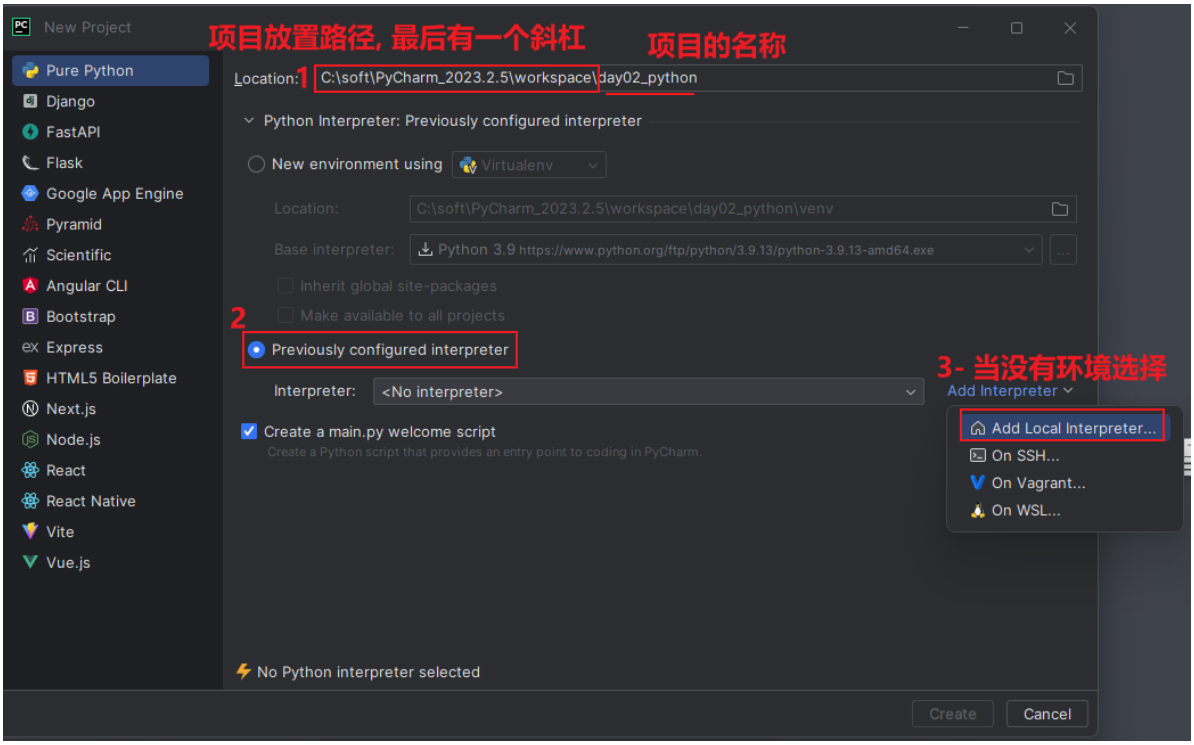
第一步：创建项目

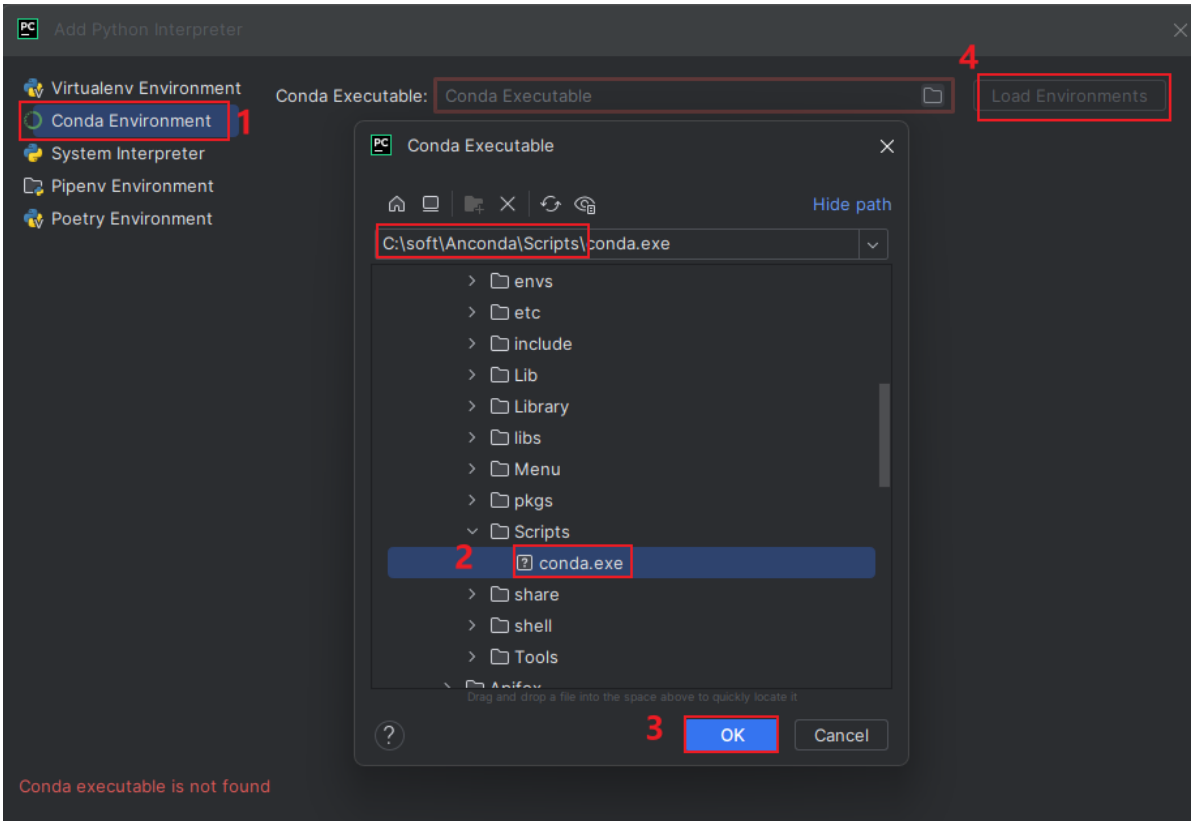


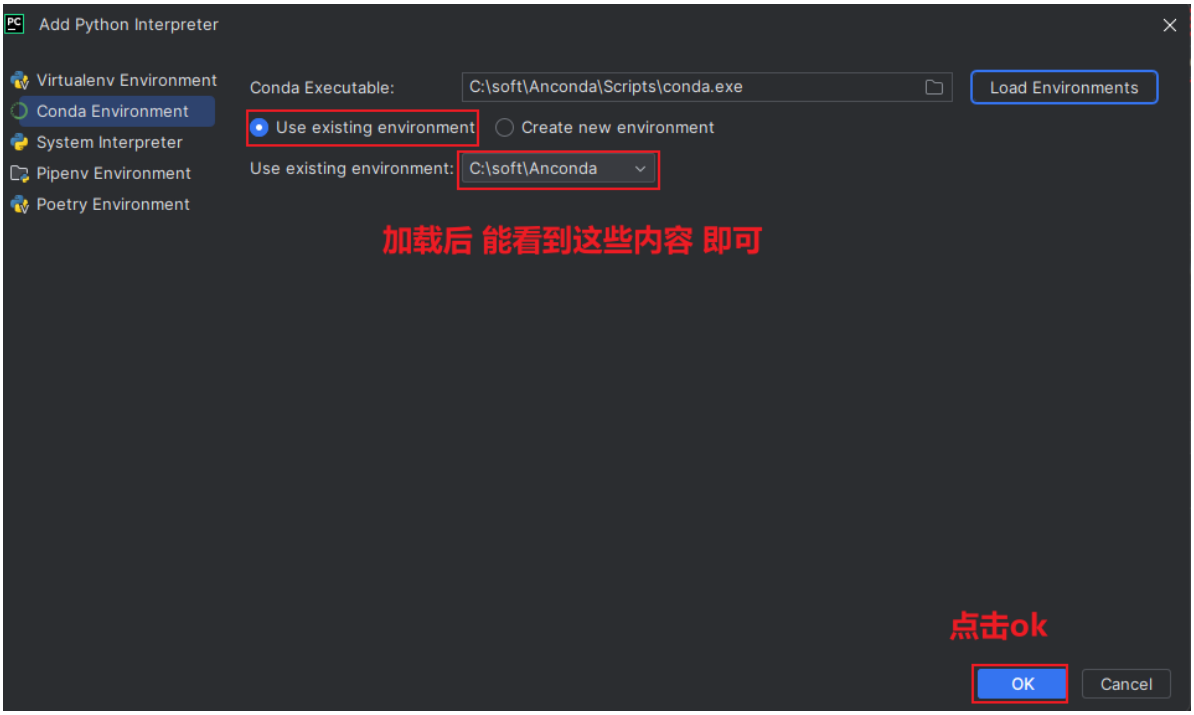
第二步：设置项目路径，必须放在C盘以外的盘符（非常重要！！！）

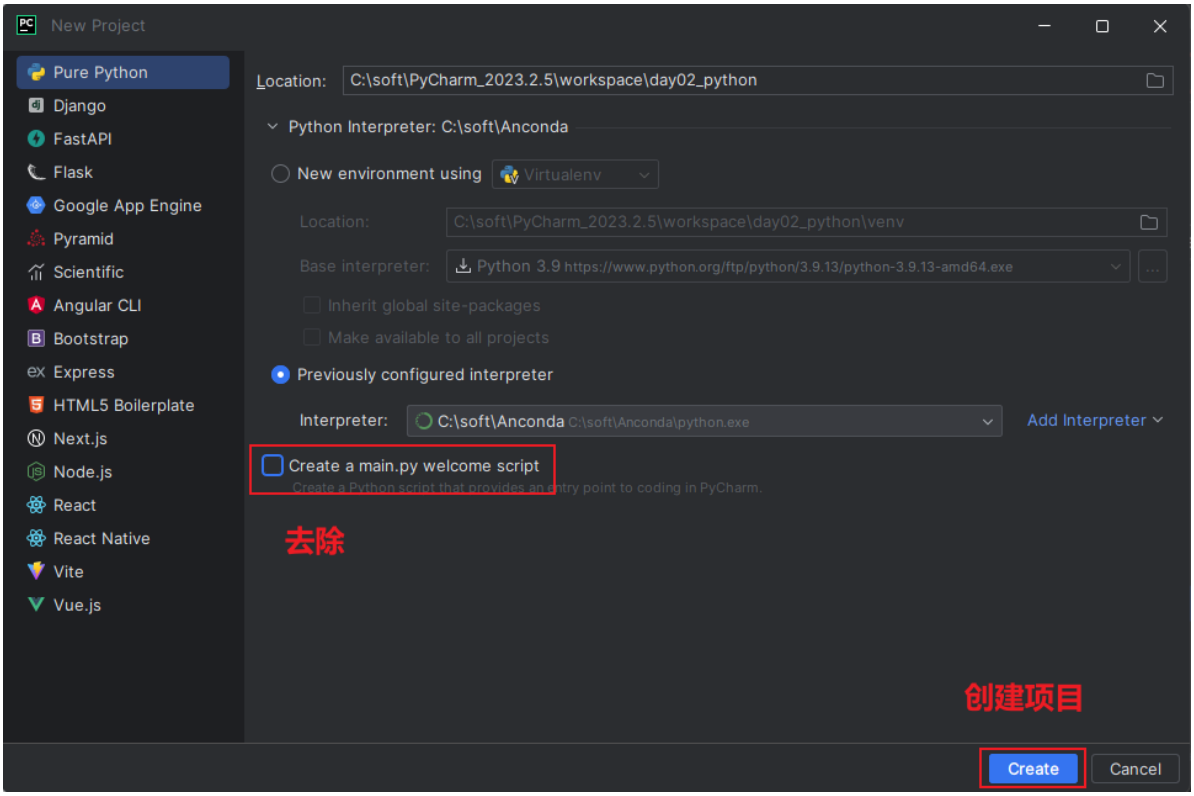


如果看不到环境：

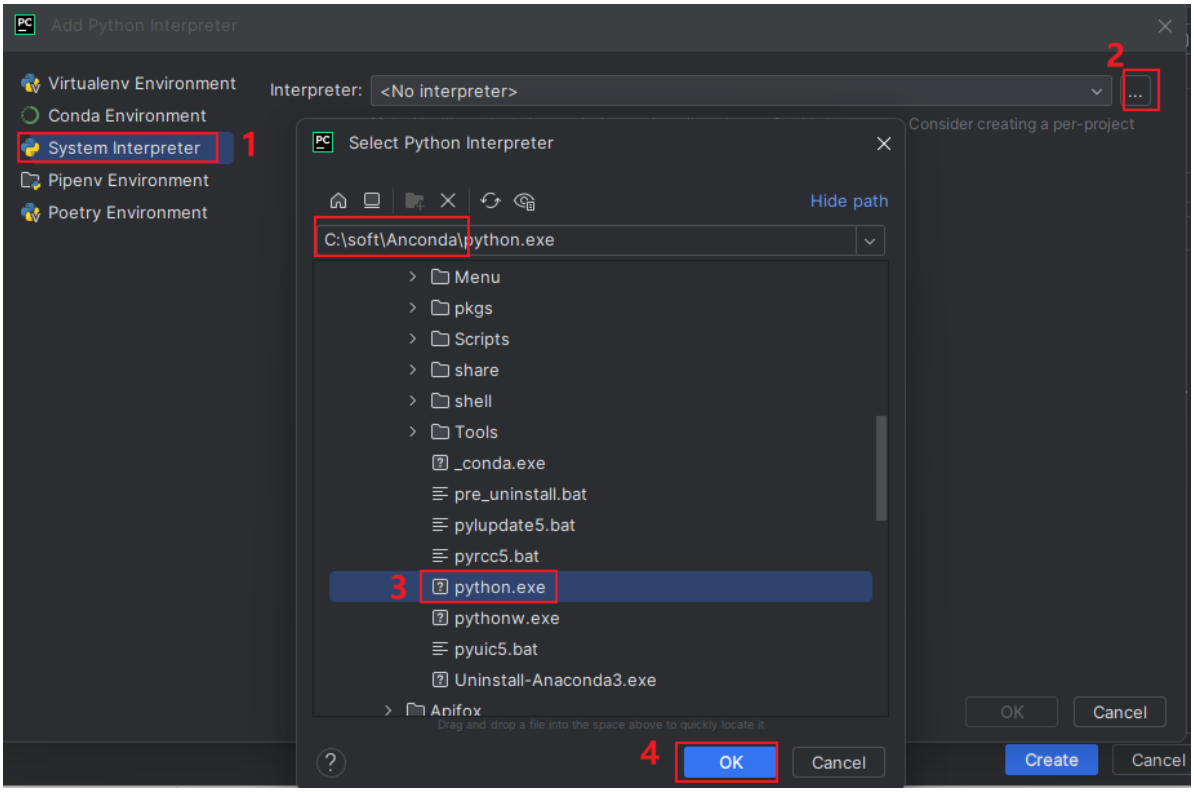

New Project
项目放置路径，最后有一个斜杠
项目的名称
Pure Python
Django
FastAPI
Flask
Google App Engine
Pyramid
Scientific
Angular CLI
Bootstrap
Express
HTML5 Boilerplate
Next.js
Node.js
React
React Native
Vite
Vue.js
Location: 1 C:\soft\PyCharm_2023.2.5\workspace\day02_python
Python Interpreter: Previously configured interpreter
New environment using Virtualenv
Location: C:\soft\PyCharm_2023.2.5\workspace\day02_python\venv
Base interpreter: Python 3.9 https://www.python.org/ftp/python/3.9.13/python-3.9.13-amd64.exe
Inherit global site-packages
Make available to all projects
2
Previously configured interpreter
Interpreter: <No interpreter>
3- 当没有环境选择
Add Interpreter
Add Local Interpreter...
On SSH...
On Vagrant...
On WSL...
Create a main.py welcome script
Create a Python script that provides an entry point to coding in PyCharm.
No Python interpreter selected
Create
Cancel


Add Python Interpreter
Virtualenv Environment
Conda Environment
1
System Interpreter
Pipenv Environment
Poetry Environment
Conda Executable: Conda Executable
4
Load Environments
Conda Executable
Hide path
C:\soft\Anconda\Scripts\conda.exe
envs
etc
include
Lib
Library
libs
Menu
pkgs
Scripts
2
conda.exe
share
shell
Tools
Drag and drop a file into the space above to quickly locate it
3
OK
Cancel
Conda executable is not found



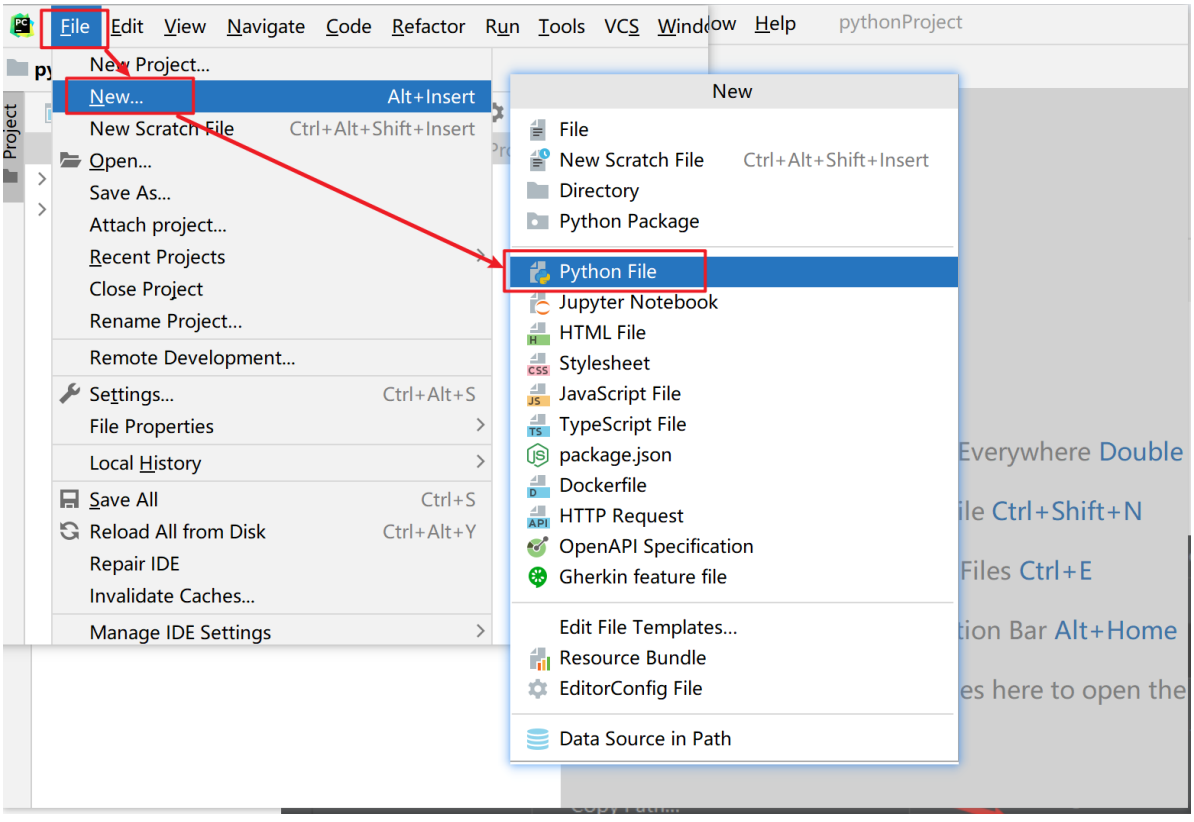


如果anaconda不能用的同学：



配置完成后，单机Create创建Python项目。

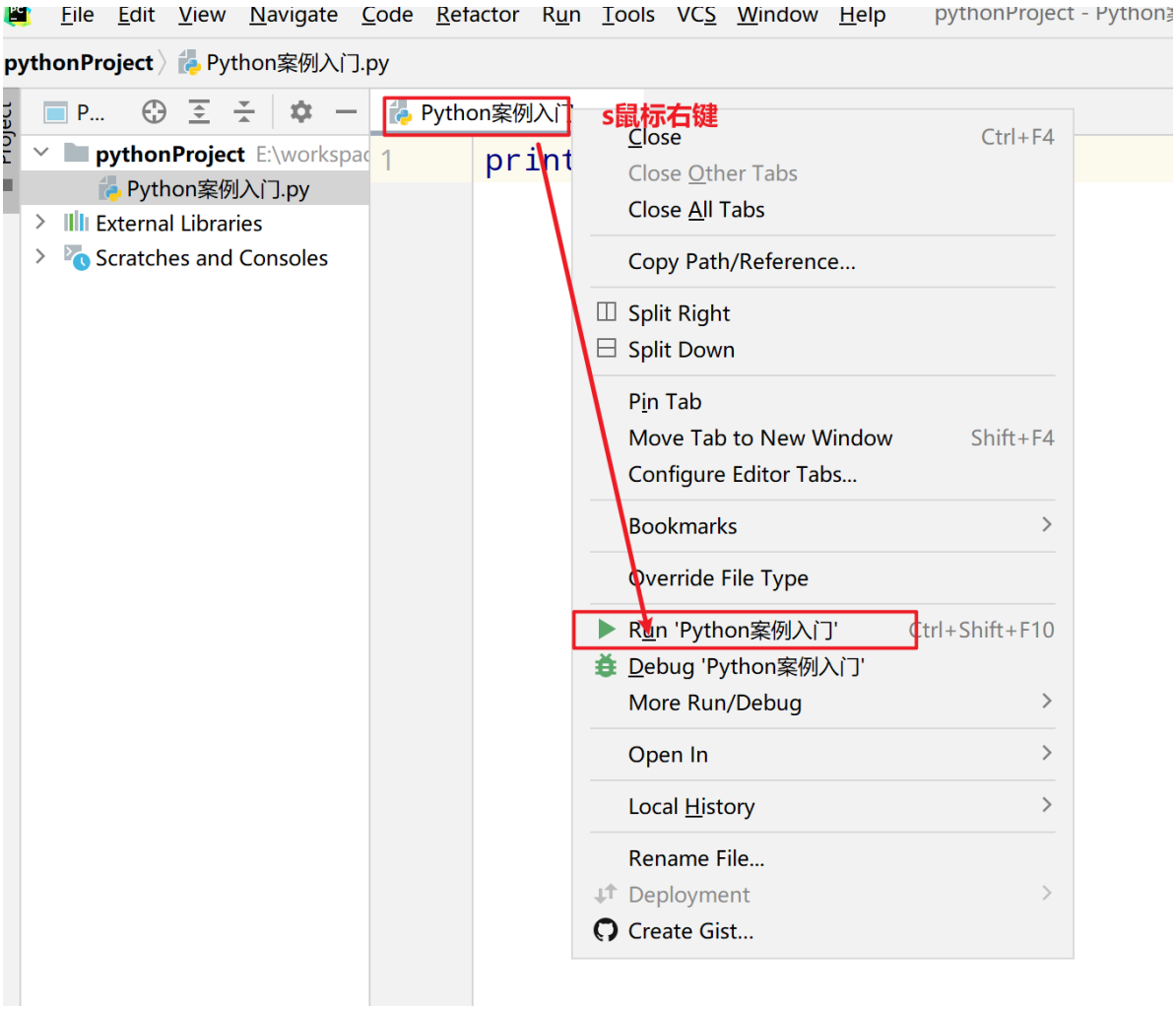
☆ 新建文件与代码书写



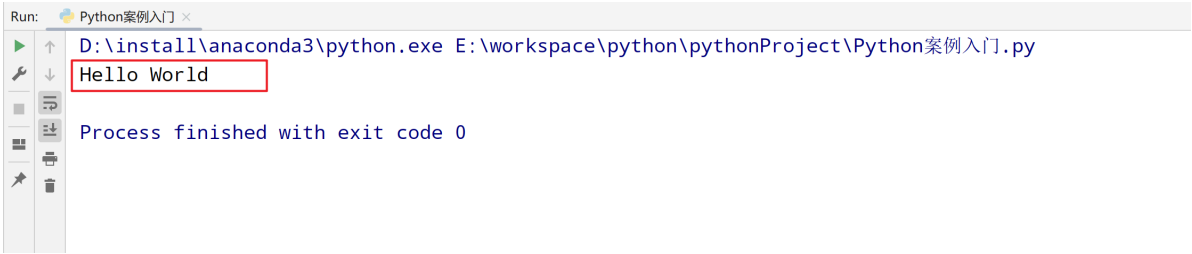
编写Hello World

```
print('Hello World')
```

☆ 运行代码

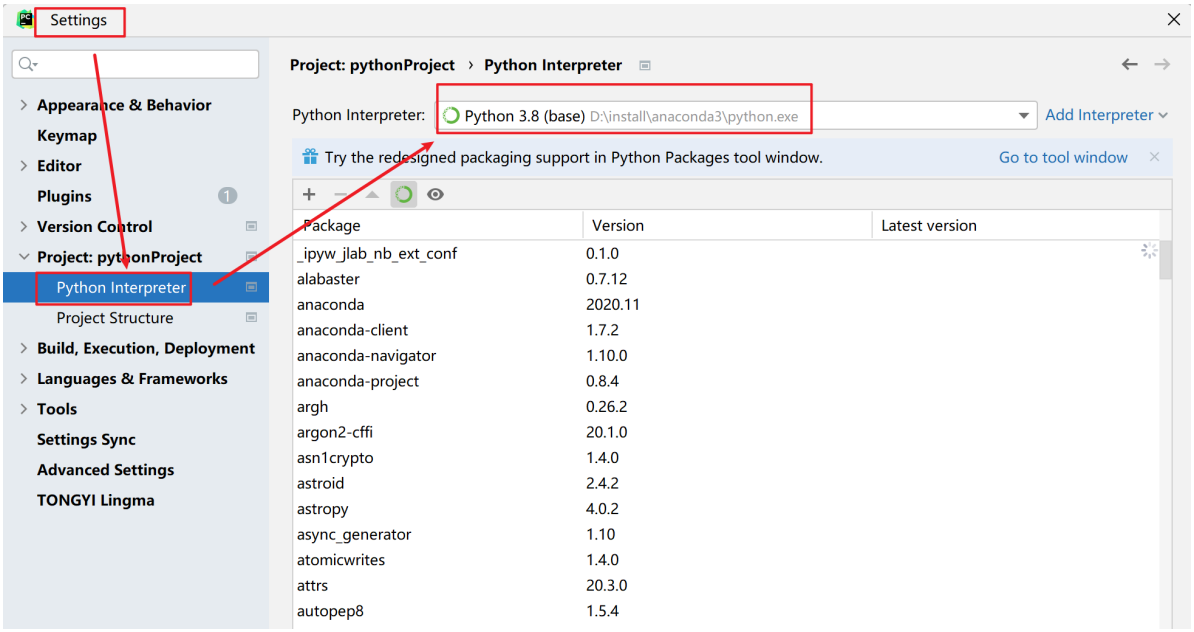


运行结果：



☆ 设置或更换Python解析器

打开File文件，找到Settings设置，如下图所示：更换Python解析器



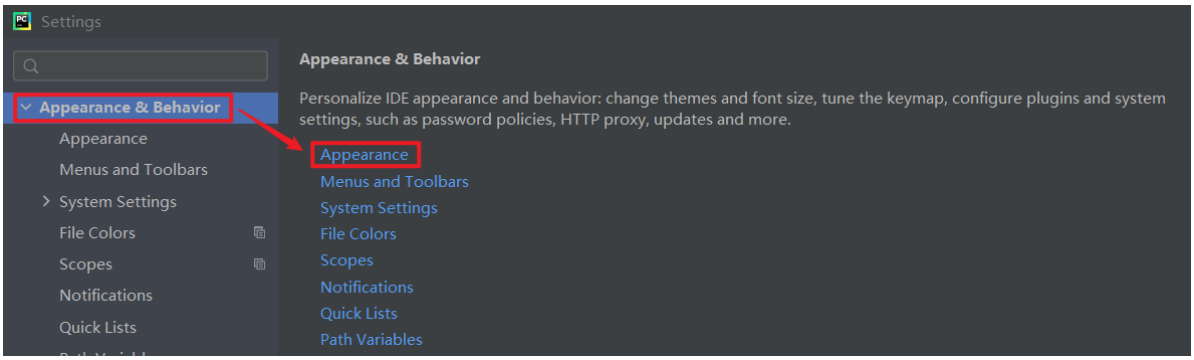
☆ PyCharm软件本身设置

① 软件主题（软件未来的样式）

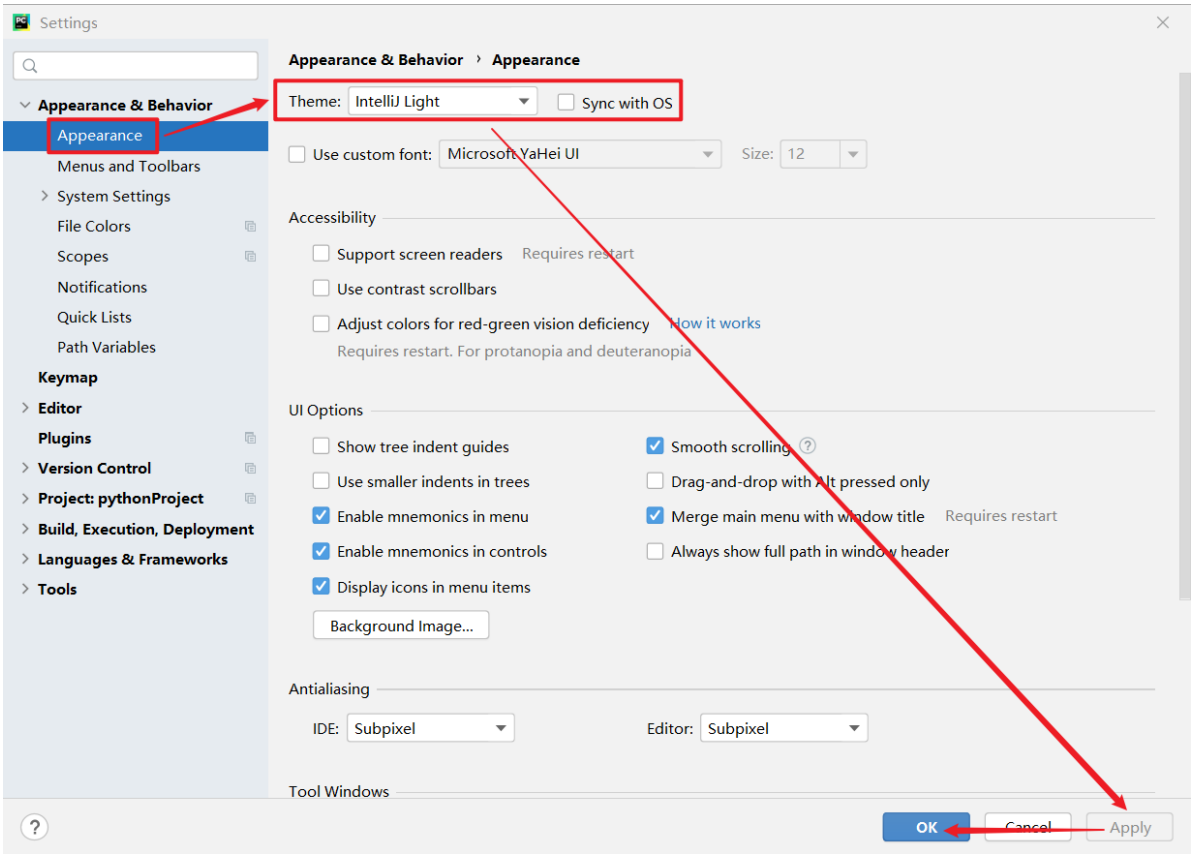
② 编码字体的设置

③ 代码字号的设置（文字大小）

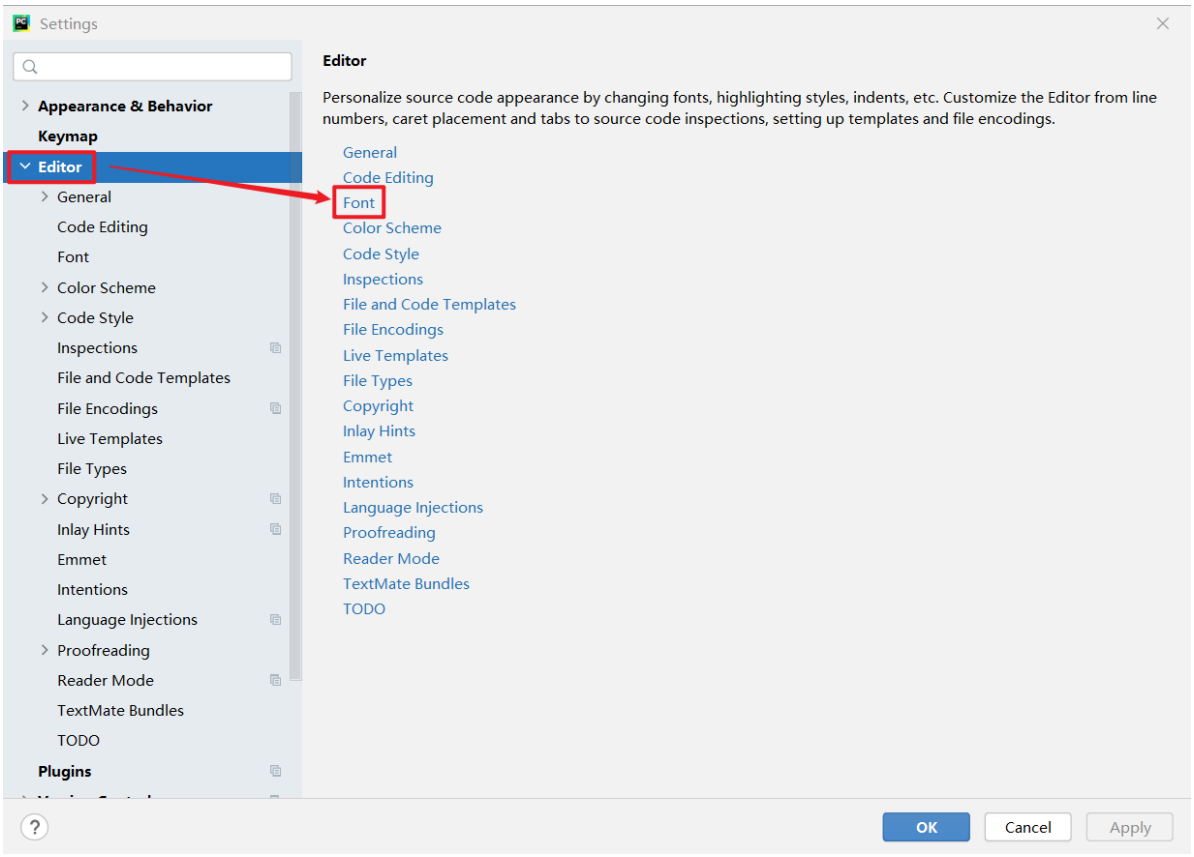
打开File文件 => Settings设置，找到界面设置：



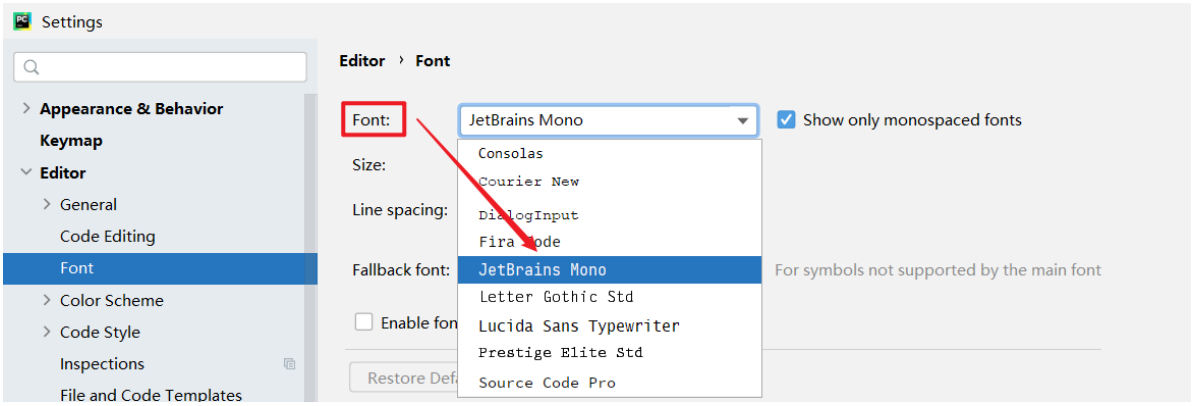
主题设置：



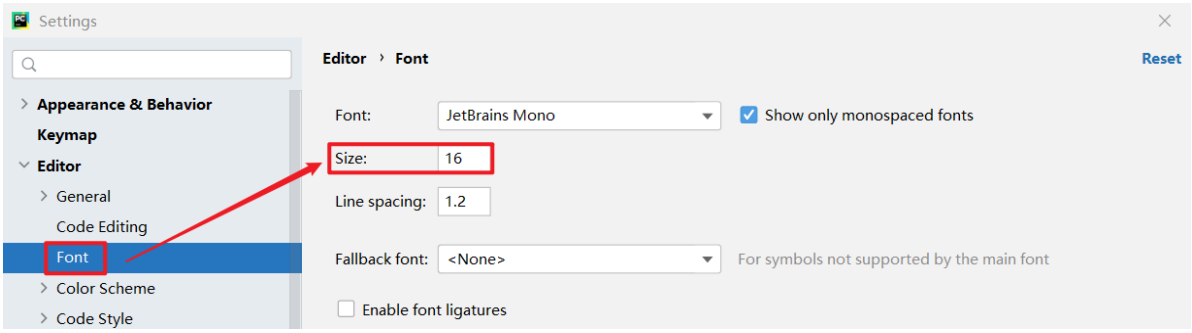
字体与字号设置：



字体设置：

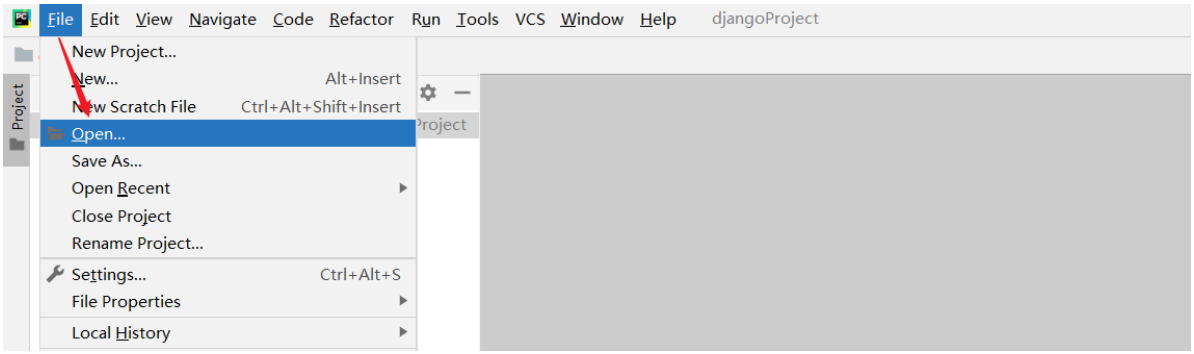


字号设置：

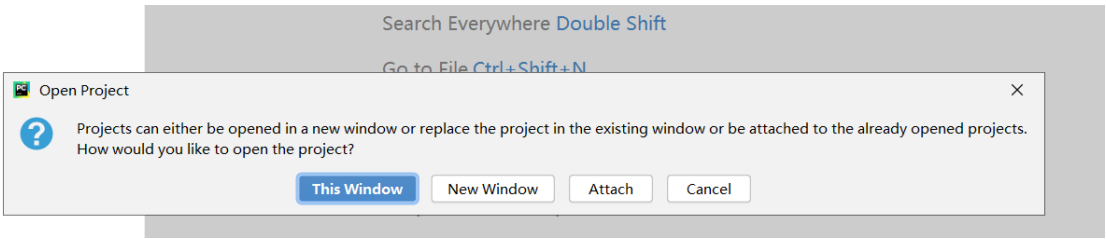


☆ 打开项目与关闭项目

打开项目：本身项目已经存在了，我们直接打开。



选择项目目录（文件夹）即可，如下图所示：

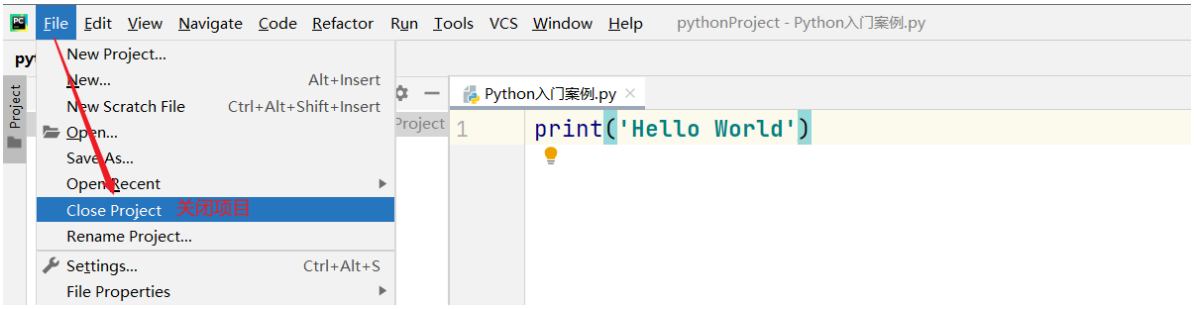


① This Window => 覆盖当前项目，从而打开目标项目

② New Window => 在新窗口打开，则打开两次PyCharm，每个PyCharm负责一个项目

③ Attach => 把两个项目合并在一起，放在同一个窗口中

关闭项目：对已经运行项目进行关闭操作。



3.【熟悉】Python注释与快捷键

3.1 注释的作用

首先强调一件事：Python代码 => Python解析器 => 机器语言，但是注释经过了Python的解释器并不会解析与执行。因为其主要就是进行代码的注释。

注释作用：提高代码的阅读性

```
    def is_win(self):
        return any(any(i >= self.win_value for i in row) for row in self.field)

    def is_gameover(self):
        return not any(self.move_is_possible(move) for move in actions)

    def draw(self, screen):
        help_string1 = '(W)Up (S)Down (A)Left (D)Right'
        help_string2 = '     (R)Restart (Q)Exit'
        gameover_string = '           GAME OVER'
        win_string = '          YOU WIN!'
        def cast(string):
            screen.addstr(string + '\n')

        def draw_hor_separator():
            line = '+' + ('+------' * self.width + '+')[1:]
            separator = defaultdict(lambda: line)
            if not hasattr(draw_hor_separator, "counter"):
                draw_hor_separator.counter = 0
            cast(separator[draw_hor_separator.counter])
            draw_hor_separator.counter += 1
```

非注释代码

```
def main(stdscr):
    def init():
        #重置游戏棋盘
        game_field.reset()
        return 'Game'

    def not_game(state):
        #画出 GameOver 或者 Win 的界面
        game_field.draw(stdscr)
        #读取用户输入得到action，判断是重启游戏还是结束游戏
        action = get_user_action(stdscr)
        responses = defaultdict(lambda: state) #默认是当前状态，没有行为就会一直在当前界面循环
        responses['Restart'], responses['Exit'] = 'Init', 'Exit' #对应不同的行为转换到不同的状态
        return responses[action]

    def game():
        #画出当前棋盘状态
        game_field.draw(stdscr)
        #读取用户输入得到action
        action = get_user_action(stdscr)
```

以#开头的就是注释

注释代码

在我们编写Python程序时，为了了提高程序的可读性，强烈建议大家为核心代码添加注释信息。

3.2 Python注释的基本语法

☆ 单行注释

单行注释，以"#"(Shift + 3)号开头，只能注释一行内容

```
# 注释内容
```

示例代码：

第一种：代码行的上面

```
# 这是单行注释 (注释可以放置到代码的上面)
print("Hello World")
```

第二种：放在代码的后面(代码后面保留2个空格)

```
print("Hello World") # 这是单行注释 (注释也可以放置到代码的后面  一般不建议)
```

☆ 多行注释

多行注释：可以同时注释多行代码或程序，常用于代码块的注释

基本语法：

```
"""
注释内容
第一行
第二行
第三行
"""
```

或

```
'''
注释内容
第一行
第二行
第三行
'''
```

课上演示：

```
# 演示注释
# 这是一个单行注释
# 格式化代码: ctrl + alt + l
# 快速复制一行并粘贴到下一行: ctrl + d
# 快速添加注释:  ctrl + /
print('Hello World')  # 注意：在代码语法中,所有的涉及到的符号, 都是英文格式的符号
"""
    我是一个多行注释
    我是一个多行注释
    我是一个多行注释
    我是一个多行注释
    我是一个多行注释
    我是一个多行注释

"""

'''
    我是一个多行注释
    我是一个多行注释
    我是一个多行注释
    我是一个多行注释
    我是一个多行注释
    我是一个多行注释
    我是一个多行注释

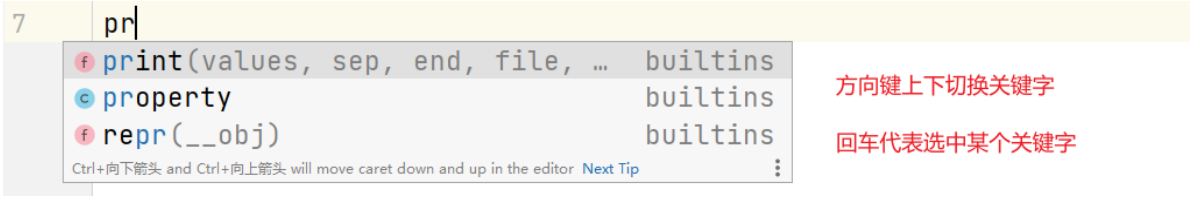
'''
```

☆ PyCharm注释小技巧

在PyCharm中，我们可以使用`Ctrl + /斜杠`来对代码或程序进行快速注释。

3.3 代码提示

在PyCharm中，当我们输入Python关键字中的前2~3个字符，其会自动进行代码提示。这个时候，我们只需要按回车即可以快速的输入某个内容。



3.4 代码保存

编写代码时，一定要养成一个好的习惯，使用`Ctrl + S`快速对代码进行保存操作。

个人建议，当写完一行代码时，就按一次。

3.5 撤销与恢复

如果不小心删除了某行代码，这个时候我们可以快速按`Ctrl + Z`就可以快速进行恢复。每按一次就撤销一次，如果撤销多了，怎么办?

答：还可以通过`Ctrl + Y`进行恢复操作

3.6 格式代码

如果写的代码，很乱，不方便阅读，那就有事没事儿就按`Ctrl + ALT + L`格式化代码

3.7 快速复制代码

如果光标所在行代码，下面要使用多次，可以按`Ctrl + D`快速复制光标所在行代码

4 【掌握】Python中的变量

4.1 引入变量的概念

那什么是变量呢?

① 变量是存储数据的容器

② 变量在程序运行过程中是可以发生改变的量

③ 变量存储的数据是临时的

4.2 变量的定义

基本语法：

```
变量名称 = 变量的值
注：等号的两边都要保留一个空格，其实Python中建议符号的两边尽量都要保留一个空格
```

> 说明：在Python程序中，这个等号和日常生活中的等号不太一样，其有一个专业名词：赋值运算符，其读法：要从右向左读，把变量的值通过 = 赋值给左边的变量。

4.3 变量的命名规则

标识符命名规则是Python中定义变量名称时一种命名规范，具体如下：

① 由数字、字母、下划线(_)组成

② 不能数字开头

③ 严格区分大小写

④ 不能使用内置关键字作为变量名称

False	None	True	and	as	assert	break	class
cotinue	def	del	elif	else	except	finally	for
from	global	if	import	in	is	lambda	nonlocal
not	or	pass	raise	return	try	while	with
yield							

举个栗子：

① abc、abc123、_abc、hello（合理）

② 123abc、@abc、abc-123（不合理）

③ _（下划线） => 请问这可以是一个变量名称么？答：可以

```
for _ in range(10):
    ...
```

> 注意：在Python变量命名时，不建议使用_开头，因为其与后面要学习的私有属性想冲突

④ 变量abc和变量ABC是同一个变量么？答：不一样，这是两个完全不同的变量

⑤ 记不住Python关键字怎么办？答：借助于help()方法

```
>>> help('keywords')
```

4.4 推荐变量的命名规则

① 变量命名一定要做到见名知义。

② 大驼峰：即每个单词首字母都大写，例如： MyName 。

③ 小驼峰：第二个（含）以后的单词首字母大写，例如： myName 。

④ **下划线：例如： my_name 。**

4.5 变量的定义与调用

在Python中，记住：变量一定要先定义，后使用，否则会报错。

定义：

```
name = 'itheima'
address = '北京市顺义区京顺路99号'
```

调用：

```
print(name)
print(address)
或
print(name, address)
```

相关的演示:

```
# 演示变量
"""
    定义变量的格式:
        变量的名称 = 值

        指的: 将等号右侧的值 赋值给 左侧的变量

    变量的名称定义规范:
        强制规范:
            1- 变量必须是以 数字 字母 或 下划线 组成
            2- 变量不能以数字开头
            3- 变量严格区分大小写
            4- 不能使用python中关键词作为变量名称

        推荐规范:
            1- 变量命名必须是见名知意
            2- 当变量名称涉及到多个单词的时候:
                方式一: 大驼峰命名     例如: MyName
                方式二: 小驼峰命名     例如: myName
                方式三:  下划线连接    例如:  my_name


"""

# 1- 定义一个年龄的变量 值为 18
age = 18

# 2- 定义一个名字的变量  值为 张三
userName = '张三'

# 如何使用变量呢?
# 如何调用: 通过变量名称
print(age)
print(userName)
```

4.6 变量的定义与使用常见问题

① 变量与字符串如何区别：

在Python中，如果要赋值的内容添加了单引号或者双引号，其就是Python中的一种数据类型：叫做字符串（日常生活中的文本信息）

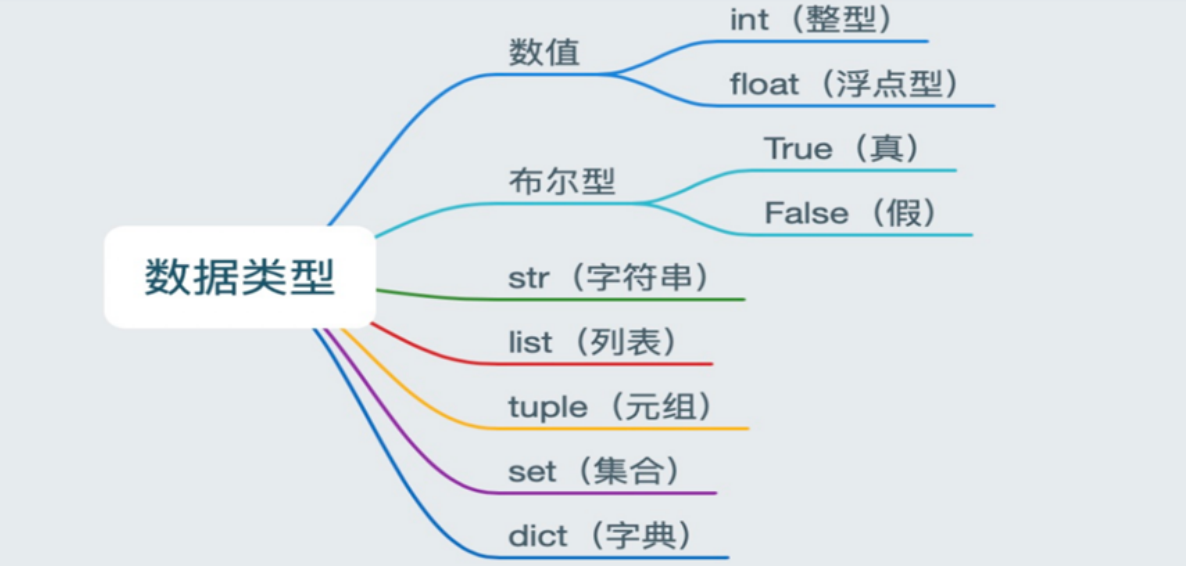
② print打印变量时，喜欢为其添加引号

```
print(name)  # 输出变量name对应的值
与
print('name')  # 输出'name'这个字符串
```

③ PyCharm快捷键 => `Ctrl + Alt + L` => 代码格式化

4.7 变量的数据类型

变量的定义非常的简单，但是很多小伙伴可能会想：变量除了存储这种字符类型的数据以外，还能存储其他类型的数据么？其实，在 Python中，我们为了应对不同的业务需求，也会把数据分为不同的类型，如下图所示：



面试题：请手写出Python中的7种数据类型？

答：数值类型、布尔类型、字符串类型、列表类型、元组类型、集合类型、字典类型

今天我们只需要了解前3种即可。

问题：如何判断一个变量到底是什么类型？

答：① 使用type(变量名称)方法，返回变量的数据类型 ② isinstance(变量名称,数据类型)，只能返回True或False（真的还是假的）

4.7.1 数值类型

数值类型就是我们日常生活中的数字，数字又分为两种形式：整数 与 小数（带小数点）

整数类型：int类型

小数类型：float类型

案例1：定义一个人的信息，姓名：Tom、年龄18岁 并且打印出结果

```
# 演示 变量的数据类型
"""
    数据类型共计有 7 种:
        数值类型 :  整数类型 int   和  小数类型 float
        布尔类型 :  真 True  假 False
        以下为五大容器,用于一次性保存多个数据:
            字符串类型:  文本数据, 一般使用 引号 引起来, 可以单引号 也可以 双引号
            列表类型
            元组类型
            集合类型
            字典类型

    如何判断数据类型:
        type(变量)  返回 对应变量的数据类型
        isinstance(变量名称,数据类型)  判断对应变量是否是这个数据类型, 如果是 返回True 否则返回False

"""
# 演示 数值类型
# 案例1：定义一个人的信息，姓名：Tom、年龄18岁  并且打印出结果
name = 'Tom'
age = 18
score = 39.5

# 打印结果
print(name)
print(type(name))

print(age)
print(type(age))

print(score)
print(type(score))

# 判断 年龄 是不是 一个整数呢?
print(isinstance(age, int))  # True

# 判断 成绩 是不是 一个整数呢?
print(isinstance(score, int))  # False
```

案例2：定义一个超市收银系统，写入一个名称：大白菜，价格：3.5 并且打印出结果

```
# 案例2：定义一个超市收银系统，写入一个名称：大白菜，价格：3.5 并且打印出结果
# 定义变量
product_name = '大白菜'
product_price = 3.5
```

```
# 打印各个变量的内容
# 将product_name打印到控制台
print(product_name)
# 将product_price打印到控制台
print(product_price)

# 获取变量的数据类型
# 获取product_name这个变量的类型是什么:
print(type(product_name))
# 获取product_price这个变量的类型是什么:
print(type(product_price))

# 判断变量的数据类型
print(isinstance(product_name, str))  # True
print(isinstance(product_price, int))  # False  因为价格是小数类型
```

4.7.2 布尔类型

布尔类型是与逻辑相关一种数据类型，只有两个值：True（真）与False（假）

案例1：手工定义一个flag变量，其值为True

```
# 演示布尔类型：  True  False
# 案例1：手工定义一个flag变量，其值为True
flag = True

# 打印变量
print(flag)
# 类型获取:
print(type(flag))  # <class 'bool'>  boolean
print(isinstance(flag,bool))  # 返回True
```

4.7.3 字符串类型

在Python变量定义中，如果其赋值的内容是通过单引号或双引号引起来的内容就是字符串str类型。

```
# 定义变量
product_name = '大白菜'

print(type(product_name)) # str 类型
```

5.【掌握】Python的输入与输出

5.1 格式化输出

目前为止，我们所有的输出都是直接通过print(变量名称)形式直接打印的。但是实际工作中，我们可能需要对变量的输出进行格式化操作（按照一定格式进行输出）。

变量的格式化

5.2 【了解】百分号格式化输出

基本语法：

```
...
print(变量名称)
print('字符串%格式' % (变量名称))
print('字符串%格式 %格式 %格式' % (变量名称1, 变量名称2, 变量名称3))
```

%格式常见形式如下：

格式符号	转换
%s	字符串
%d	有符号的十进制整数
%f	浮点数
%c	字符
%u	无符号十进制整数
%o	八进制整数
%x	十六进制整数（小写ox）
%X	十六进制整数（大写OX）
%e	科学计数法（小写'e'）
%E	科学计数法（大写'E'）
%g	%f和%e的简写
%G	%f和%E的简写

案例：定义两个变量name='itheima', age=18，按照如下格式进行输出：我的名字是itheima，今年18岁了。

```
# 1- 百分号格式的输出
'''
    语法结构:
        print( '字符串信息.... %格式  %格式  %格式 ' % (变量1,变量2,变量3....) )

        说明: 在字符串中 引用变量 可以直接和字符串拼接在一起输出打印

        %格式:
            %s  字符串
            %d  有符号的整数
            %f  小数类型
'''

# 案例：定义两个变量name='itheima', age=18，按照如下格式进行输出：我的名字是itheima，今年18岁了。
# 定义变量
name = 'itheima'
```

```
age = 18

# 输出操作
print('我的名字是%s，今年%d岁了。' % (name, age))
```

案例：定义两个变量title='大白菜'，price=3.5，按照如下格式进行输出：今天蔬菜特价了，大白菜只要3.5元/斤。

```
# 案例：定义两个变量title='大白菜'，price=3.5，按照如下格式进行输出：今天蔬菜特价了，大白菜只要3.5元/斤。
# 定义变量:
title = '大白菜'
price = 3.5

# 输出变量: 百分号方式输出
print('今天蔬菜特价了，%s只要%.1f元/斤。' %(title,price))
```

5.3 【了解】format方法格式化输出

大括号的书写：shift + p右边的二个键

基本语法：

```
...
print('字符串{}'.format(变量名称1))
print('{}字符串{}'.format(变量名称1，变量名称2))
```

案例：定义两个变量，name='孙悟空'，mobile='18878569090'，按照以下格式进行输出"姓名：孙悟空，联系方式：18878569090"

```
# 使用 format格式输出
'''
    语法结构:
        print('{}字符串信息{}'.format(变量1,变量2...))
'''
# 案例：定义两个变量，name='孙悟空'，mobile='18878569090'，按照以下格式进行输出"姓名：孙悟空，联系方式：18878569090"
# 定义变量
name = '孙悟空'
mobile = '18878569090'

# 输出操作
print('姓名：{}，联系方式：{}'.format(name,mobile))

# 针对format 提供更为精简的简化方案
print(f'姓名：{name}，联系方式：{mobile}')
```

5.4 【重要】format方法简写形式格式化输出（推荐）

在Python3.6以后版本，为了简化format输出操作，引入了一个简写形式：

```
# 案例：定义两个变量，name='孙悟空'，mobile='18878569090'，按照以下格式进行输出"姓名：孙悟空，联系方式：18878569090"
# 定义变量
name = '孙悟空'
mobile = '18878569090'

# 针对format 提供更为精简的简化方案
print(f'姓名：{name}，联系方式：{mobile}')
```

完整的演示的代码操作：

```
# 演示输出：
# 1- 百分号格式的输出
'''
    语法结构：
        print( '字符串信息.... %格式  %格式  %格式 ' % (变量1,变量2,变量3....) )

        说明：在字符串中 引用变量 可以直接和字符串拼接在一起输出打印

        %格式：
            %s  字符串
            %d  有符号的整数
            %f  小数类型
'''

# 案例：定义两个变量name='itheima', age=18，按照如下格式进行输出：我的名字是itheima，今年18岁了。
# 定义变量
name = 'itheima'
age = 18

# 输出操作
print('我的名字是%s，今年%d岁了。' % (name, age))
# 基于format的简写方式：
print(f'我的名字是{name}，今年{age}岁了。')

# 案例：定义两个变量title='大白菜'，price=3.5，按照如下格式进行输出：今天蔬菜特价了，大白菜只要3.5元/斤。
# 定义变量：
title = '大白菜'
price = 3.5

# 输出变量：百分号方式输出
print('今天蔬菜特价了，%s只要%.1f元/斤。' % (title, price))
# 基于format的简写方式：
print(f'今天蔬菜降价了,{title}只要{price}元/斤.')

# 使用 format格式输出
'''
    语法结构：
        print('{}字符串信息{}'.format(变量1,变量2...))
```

```
'''
# 案例：定义两个变量，name='孙悟空'，mobile='18878569090'，按照以下格式进行输出"姓名：孙悟空，联系方式：18878569090"
# 定义变量
name = '孙悟空'
mobile = '18878569090'

# 输出操作
print('姓名：{}，联系方式：{}'.format(name,mobile))

# 针对format 提供更为精简的简化方案
print(f'姓名：{name}，联系方式：{mobile}')
```

5.5 格式化输出中的转义符号

在字符串中，如果出现了\t和\n，其代表的含义就是两个转义字符

```
\t ：制表符，一个tab键（4个空格）的距离
\n ：换行符
```

相关的操作:

```
# 换行符 \n 和  制表符 \t
# print操作，默认的结束符号为 \n，如何调整的默认结束符号呢？在print 添加一个 end参数
print('学AI,就来黑马程序员~~~')
print('学AI,就来黑马程序员~~~',end='\t')
print('学AI,就来黑马程序员~~~',end='\n')

print('学AI,就来\n\n黑马\t\t程序员~~~')
```

5.6 为什么需要输入

到目前为止，我们所有的程序都只能把数据输出给用户。但是实际工作中，我们经常输入获取用户的输入信息，如银行系统中的密码输入、淘宝中的用户登录验证。



5.7 input() 输入方法

在Python中，如果想让Python程序接受用户的输入信息，可以使用input()方法

基本语法：

```
input()
```

但是往往只有input()方法，其意义不大，我们还应该使用一个变量来临时接受用户的输入，已方便后期的操作。

```
变量名称 = input('提示信息：')
```

案例：银行系统中的，输入密码的过程: 提示内容　请输入您的密码？　并且打印出用户输入的内容

```
# 输入操作：
"""
    语法结构：
        变量 = input('提示信息内容...')

        指的：将用户输入的内容，通过变量接收

        注意：用户输入的内容，都是字符串类型的数据

"""
# 案例：银行系统中的，输入密码的过程：提示内容 请输入您的密码？　并且打印出用户输入的内容
password = input('请输入您的密码:')  # 程序执行到这一行代码的时候，会阻塞住，等待用户的输入，用户输入完成后 才会执行下面的代码
# 打印用户输入的内容：
print(f"用户输入的内容为:{password}")

# 查看变量的数据类型
print(type(password))  #  <class 'str'>

# 需求：请键盘录入二个分别为用户名 和 密码 ，在录入的时候，给用户做出提示，例如 请输入您的用户名　并且将结果使用一个print打印出
# 1- 使用 input 实现键盘录入二个内容
```

```
userName = input('请输入您的用户名:')
password = input('请输入您的密码:')

# 2- 打印输出: 使用一个print语句输出
print(f"输入的用户名为:{userName},输入的密码为:{password}")
```

5.8 input()方法重要事项

记住：所有由input()方法获取的数据都是“字符串”类型

```
name = input('请输入您的姓名：')
age = input('请输入您的年龄：')

print(type(name))  # <class 'str'>
print(type(age))  # <class 'str'>
```

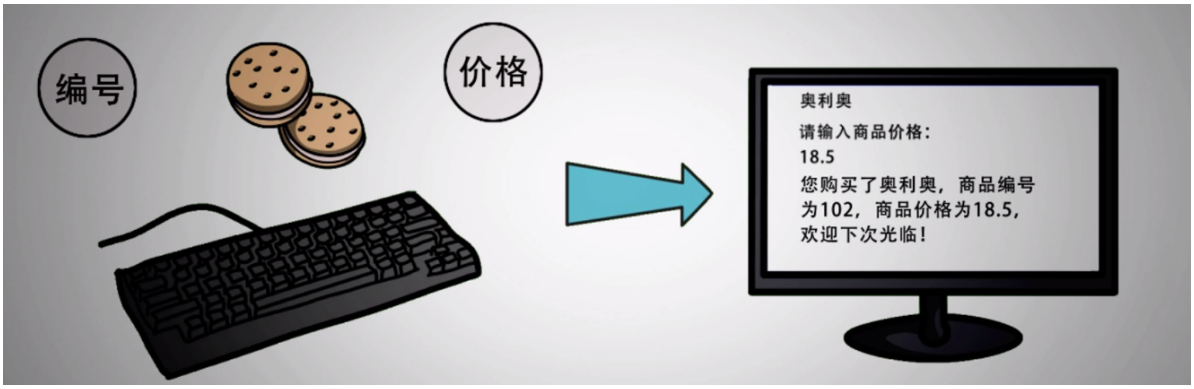
总结：

① input()可以用于接收由外部设备输入的信息，但是如果用户没有输入任何内容，则input()函数会中止当前代码的继续执行，处于等待状态，直到用户输入结束。

② 所有由input()方法获取的数据都是“字符串”类型

6.【掌握】Python数据类型转换

6.1 使用Python实现超市的收银系统



Python：

```
name = input('请输入您要购买商品名称：')
id = input('请输入您要购买商品编号：')
price = input('请输入您要购买的商品价格：')

print(f'您购买了{name}，商品编号为{id}，商品价格为{price}，欢迎下次光临！')
```

以上程序虽然可以按照上图完成程序的正常输出，但是遗留了一个非常严重的问题：这个价格price变量无法参与数学运算（如买了两个奥利奥，应该是18.5 * 2），原因在于input()方法返回的所有的结果都是str字符串类型。

怎么解决以上问题呢？ 答：使用数据类型转换

6.2 了解Python数据类型的转换方法

函数	说明
int(x)	将x转换为一个整数
float(x)	将x转换为一个浮点数
complex(real [,imag])	创建一个复数，real为实部，imag为虚部
str(x)	将对象 x 转换为字符串
repr(x)	将对象 x 转换为表达式字符串
eval(str)	用来计算在字符串中的有效Python表达式,并返回一个对象
tuple(s)	将序列 s 转换为一个元组
list(s)	将序列 s 转换为一个列表
chr(x)	将一个整数转换为一个Unicode字符
ord(x)	将一个字符转换为它的ASCII整数值
hex(x)	将一个整数转换为一个十六进制字符串
oct(x)	将一个整数转换为一个八进制字符串
bin(x)	将一个整数转换为一个二进制字符串

```
# 演示数据类型转换
"""
    数据类型转换的操作:
        int(x) : 将 x 转换为 整数类型
        float(x): 将 x 转换为 小数类型
        str(x): 将 x 转换为 字符串类型

    注意:
        在转换的时候, 需要转换的值应该是什么类型, 就转换为对应的类型
"""
# 演示其他的类型转换:
a = '10'
b = '3.5'
e = 20

# 将 a 转换为 整数类型  :
c = int(a)
print(c)
print(type(c))

# b 转换为 小数类型
d = float(b)
print(d)
print(type(d))

# 思考 如果将b 转换为 整数类型呢?  无法转换  因为 b 本省就不是整数类型
# print(int(b))
```

```
b = 3.5
print(int(b)) # 不会报错， 但是会将小数位丢失，仅保留整数位


# e 转换为 字符串类型
f = str(e)
print(f)
print(type(f))



# 演示数据类型转换
name = input('请输入您要购买商品名称：')
id = input('请输入您要购买商品编号：')
price = input('请输入您要购买的商品价格：')

print(f'您购买了{name}，商品编号为{id}，商品价格为{price}，欢迎下次光临！')


# 演示数据类型转换
name = input('请输入您要购买商品名称：')
id = input('请输入您要购买商品编号：')
price = input('请输入您要购买的商品价格：')

print(f'您购买了{name}，商品编号为{id}，商品价格为{price}，欢迎下次光临！')


# 需求： 价格需要再原有的基础上涨价 10%

# 类型转换操作:
price = float(price)


price = price * 1.1
print(f'您购买了{name}，商品编号为{id}，商品价格为{price:.2f}，欢迎下次光临！')
```

相关案例:

```
# 类型转换案例
# 需求: 请键盘录入一个商品名称，商品的价格，商品的数量，输出： 您购买了 xxx ，商品的单价为
xxx,您购买了xx个，总价为 xxx
# 键盘录入： input('提示词')
# 注意： 通过键盘录入的内容， 其数据类型都是字符串

# 1- 通过键盘录入三个信息:
pName = input('请输入商品名称:')
price = input('请输入商品价格:')
num = input('请输入商品数量:')

# 2- 计算 商品的总价

# 2.1 类型转换操作： price 和 num
price = float(price)
num = int(num)
# 2.2 计算总价
total = price * num
```

```
# 3- 输出结果
print(f'您购买了{pName} , 商品的单价为{price},您购买了{num}个, 总价为{total}')
```